GLUTTONS

FRONTEND FINAL IMPLEMENTATION MANUAL

Canonical handoff for Carlos. Rewrites the original Frontend Integration Manual using the final September 2026 Gluttons rules, frontend architecture, Curtis test findings, and all UI/UX changes implemented through the final frontend iterations.

AUTHORITY This manual does NOT embed Solidity. The deployed ABI/contracts + Developer Architecture & Security Spec v1.2 are contract authority. The frontend must never invent or override gameplay rules.

CANONICAL SOURCE	ROLE
Final Design Bible v1.3	Gameplay behavior / policy
Final Mathematical Lock v1.3	Constants / formulas
Developer Architecture & Security Spec v1.2	Implementation / security
Frontend Blueprint FINAL V02	Frontend architecture / UX
Frontend Rules v1.1	Player-facing rules
Website Messaging v2.1	Stage copy / microcopy
Monte Carlo v1.4	REVALIDATION REQUIRED; not frontend authority

SUPERSEDES Replace the old 49-page Frontend Integration Manual as implementation guidance. Do not copy its embedded GameEngine/Inspector Solidity because it contains obsolete cooldown, 12H protection, test-time constants, and stale phase logic.

01 / NON-NEGOTIABLES

THE FRONTEND CONTRACT

These are the rules Carlos must not reinterpret while building the final UI.

RULE	FINAL FRONTEND REQUIREMENT
Chain authority	Onchain state is canonical. React may optimistically animate, never decide validity.
Supply	Dynamic. Mainnet target 2,000; Curtis may be 100. Never hardcode 2,000 for inventory/matrix discovery.
Game Start	s_gameStart > 0 is a one-way LIVE latch. Mint/PreMint UI never returns.
Poison	NO attacker cooldown. Attacker >1H. Normal target >1H. Attacker -1H. Normal target shield 10H. Faster -> Final Bite 1H.
FAST at 0H	Still alive pre-LS. Poison can trigger Final Bite.
Death	Logical death immediately becomes Fresh. Reap is infrastructure, not player gameplay.
Freshness	Fresh -> Rotten after 24 spoil-hours. Fridge slows spoilage 4:1 for 24H; never resets freshness.
Phases	Feast -> Plague -> LS Warning -> Last Supper -> Settled; never backward.
Last Supper	Feed / Fast / Poison / Power OFF permanently. Fresh / Rotten / Devour / Truce remain.
Disabled actions	Always show human reason; never only a grey button.
Responsive	Zero horizontal PAGE scroll from 320px to ultrawide.

POISON COPY Use SHIELD DOWN = vulnerable and SHIELD UP = protected. There is no cooldown timer, label, read, preflight, or UI state.

02 / STACK

PROJECT ARCHITECTURE

Keep the existing stack. The final pass is a state-sync and product-readability problem, not a framework rewrite.

LAYER	CHOICE	NOTES
Framework	Next.js App Router	Use src/ directory structure.
Language	TypeScript	Strict types around ABI reads and token states.
Styling	TailwindCSS + global CSS	Terminal/degen protocol aesthetic; mobile-first.
Wallet / Web3	Wagmi + RainbowKit + Viem	Read/write/simulate + receipts.
Global State	Jotai	Macro protocol state + stage latch.
Images / Metadata	GluttonNFT.tokenURI	Hydrate lazily; do not block first inventory paint.
RPC reads	Small batches / deployless multicall on Curtis	Never a single 2,000-token heavy multicall.

Recommended source layout:

PATH	RESPONSIBILITY
src/app/	Routes: /, /live, /my-gluttons, /leaderboard, /inspect, /rules, /communities
src/components/protocol/	Stage, phase, progress, transaction feedback, target controls
src/components/stadium/	Matrix, compact board, event tape, global bars
src/components/inventory/	Owned Glutton cards, corpse cards, eater selectors
src/hooks/	Protocol sync, owned discovery, token hydration, transaction refresh
src/state/	Jotai atoms: protocol, live latch, UI feedback
src/lib/	ABIs, addresses, chain config, formatters, state derivation helpers

03 / STAGES

ONE-WAY PRODUCT STAGE MACHINE

Mint is temporary. LIVE is permanent.

STAGE	SOURCE	UI
Awareness	Offchain campaign state	Follow X + wallet capture only. No wallet connect required.
Community Pre-Mint	s_gameStart==0 and s_preMintEnd==false	Community eligibility + allocations + preMint.
Public Mint	s_gameStart==0 and s_preMintEnd==true	Public mint, wallet counter, progress, rules.
LIVE	s_gameStart>0	Stadium + My Gluttons + Leaderboard + gameplay.
SETTLED	Inspector/GameEngine settled	Claim + inspect + archive; no gameplay actions.

LIVE LATCH

Initial load: read s_gameStart. If >0, immediately lock LIVE. After any mint receipt, re-read s_gameStart. Once observed, never render Mint/PreMint again during that session; localStorage may suppress flash, but chain remains authority.

Required behavior after LIVE lock:

- Stop mint-specific polling permanently.
- Redirect /mint and community claim UI away from mint actions.
- Continue gameplay polling/events and targeted token refresh.
- Do not use s_totalMinted as a reason to return to mint UI.
- Use Starting Population S for live population geometry and token scan range.

04 / DATA

GLOBAL STATE + INSPECTOR

Use one macro protocol model and focused token reads. Do not duplicate state in multiple React trees.

READ	USE
Inspector.getGlobalView	phase, aliveCount, currentMealSeconds, settled
Inspector.getTokenView(tokenId)	owner, visualState, expiry, Hungry
GameEngine token state getter	FASTING, Final Bite, poisonProtectedUntil, deadAt, spoil/fridge fields exposed by deployed ABI
GameEngine.s_gameStart	permanent LIVE latch
GameEngine.S	starting population / live scan geometry
GluttonNFT.balanceOf(wallet)	expected owned count
GluttonNFT.ownerOf(id)	ownership discovery / verification
GluttonNFT.tokenURI(id)	lazy metadata/image hydration
PrizeVault / native balances	Pot / settled claim information as available

IMPORTANT No poisonCooldownUntil read exists in the final frontend. If an ABI still contains an obsolete field on an old test deployment, ignore it in UI and migrate to the updated contracts before mainnet.

05 / ROUTES

FINAL ROUTE MAP

Keep gameplay navigation compact. Communities remains a separate pre-game/admin surface, not a main gameplay tab.

ROUTE	PURPOSE	NOTES
/	Stage-aware landing	Awareness/Mint before start; LIVE after latch.
/live	Stadium	Progress first -> matrix -> compact leaderboard.
/my-gluttons	Owner cockpit	Wallet positions + actions + corpse inventory.
/leaderboard	Full Survival Board	Status ranking only; no hunting filters.
/inspect	Neutral token due diligence	Read-only canonical public state.
/rules	Rules + action library	One-screen explanation first, details below.
/communities	Community allocation/admin	Separate URL. Close/redirect after Game Start.

Gameplay nav after LIVE: LIVE / MY GLUTTONS / LEADERBOARD / RULES / OPENSEA. Inspector is reached contextually from matrix, leaderboard, or token cards.

06 / LIVE

STADIUM INFORMATION HIERARCHY

The matrix is not the hero. The first screen must explain the state of the game before showing the population field.

1. 1. Phase / global state header.
2. 2. Progress bars: Alive, Pot, Metabolism, current Meal, next phase threshold.
3. 3. Primary navigation / contextual actions.
4. 4. Live matrix representing the full Starting Population S.
5. 5. Compact Survival Board / Leaderboard at the bottom.

STADIUM RULE Numbers, states, thresholds and events before explanatory paragraphs. A spectator should understand the pressure without connecting a wallet.

GLOBAL MODULE	DISPLAY
PHASE	THE FEAST / THE PLAGUE / LS WARNING / LAST SUPPER / SETTLED
ALIVE	Current canonical aliveCount and % of S
POT	Current prize pool
MEAL	Current Feed time value
METABOLISM	completed bars / progress to next bar / +2H floor
NEXT	Next phase trigger / warning threshold
EVENT TAPE	Recent meaningful state changes when reliable event/indexer support exists

07 / MATRIX

THE 2,000-CELL LIVE WORLD

The matrix is a living map of token IDs, not a rearranging list. On Curtis, it renders S cells (e.g. 100), not 2,000 fake cells.

STATE	MATRIX TREATMENT
Alive	alive color; stable token position
Hungry	warning color
Fasting	huntable status color
Final Bite	urgent pulse + countdown
Fresh	corpse/fresh color; still visible
Rotten	rotten color; still visible
Consumed/Burned	empty/ash state; token position remains identifiable

Interaction: hover/tap shows token ID + public state summary. Click opens neutral inspect. The matrix must not reorder on death. Death does NOT disappear because Fresh/Rotten are still game resources.

PERFORMANCE Progressive/paginated reads only. Never hit 2,000 token states in one heavy multicall. Matrix may later use an indexer, but action validity is always onchain.

08 / LEADERBOARD

SURVIVAL BOARD, NOT A HIT LIST

Leaderboard exists as a compact block at the bottom of LIVE and as full /leaderboard.

FIELD	RULE
Rank	Longest remaining clock first.
Token ID	Always visible.
Clock	Public remaining clock / DEAD / status-appropriate.
State	Alive / Hungry / Fasting / Final Bite / etc.
Shield	DOWN or UP + remaining protection.
Owner	Short address if useful.
Inspect	Allowed. Neutral read.
Poison button	Do NOT place direct Poison CTAs in leaderboard.

Forbidden filters/sorts: weakest, lowest clock, Fasting-only, vulnerable-only, best target. The official UI exposes public state but does not automate target discovery.

09 / INVENTORY

MY GLUTTONS - OWNER COCKPIT

Show real wallet-owned positions only. No sample data, no reference-image placeholders after hydration.

STEP	IMPLEMENTATION
1. Count	balanceOf(wallet) = expected owned Glutton count.
2. Range	Pre-game: minted range. LIVE: derive scan range from S / deployment supply.
3. Discover	ownerOf in small batches; keep successful IDs.
4. Hydrate	Inspector/token state in <=20 heavy batches.
5. Metadata	tokenURI lazy after card is visible.
6. Failure	If batch fails: 50 -> 20 -> 10 -> 5 -> 1. Do not erase loaded positions.
7. Diagnostics	If balance>0 and hydrated=0, show INVENTORY READ FAILED with actionable error + retry.

CURTIS	A 100-token test deployment must work without changing frontend code. Dynamic supply is mandatory.

Each living card should show: art, token ID, state, clock, shield if relevant, and only context-valid actions. Each action refreshes only affected token IDs after receipt.

10 / POISON

POISON UX - FINAL

Poison should feel like a game action, not a wallet transaction.

CHECK	UI BEHAVIOR
Attacker ownership	Only owned attacker IDs.
Attacker state	Alive, non-Fasting, non-Final-Bite, >1H.
Target self	Reject self target.
Normal target	Alive, non-Final-Bite, SHIELD DOWN, >1H.
Faster target	Can be targeted; successful Poison triggers Final Bite 1H.
Shield	DOWN = vulnerable. UP = protected + countdown.
Cooldown	NONE. Do not show/read/check one.
Preflight	simulateContract / eth_call immediately before wallet opens.

Target panel example: TARGET #0009 / LIFE CLOCK 18:58:04 / POISON SHIELD DOWN / VULNERABLE / [POISON].

SUCCESS - NORMAL	POISON SUCCESSFUL / TARGET #0009 HIT / CLOCK 18:58:04 -> 09:29:02 / SHIELD UP 10:00:00 / YOUR CLOCK -1H.
SUCCESS - FASTING	POISON SUCCESSFUL / TARGET WAS FASTING / FINAL BITE TRIGGERED / 01:00:00 / FEED OR DIE.

Generic CONFIRMED is transport feedback only and disappears. The primary CTA returns to POISON after settlement.

11 / CORPSES

CORPSE INVENTORY + FRESHNESS

Freshness is a visible resource. The player must never guess when a corpse becomes Rotten.

ELEMENT	REQUIRED DISPLAY
State	FRESH CORPSE or ROTTEN
Freshness	Permanent bar + percentage while Fresh
ROTS IN	Countdown derived from spoilage
Fridge	OFF or ACTIVE + separate effect timer
Keep Fresh	Owner-only Fresh corpse action; 0.0003 ETH / 24H mainnet
Eat Fresh	Choose eligible owned Hungry eater; +12H, cap 36H
Eat Rotten	Only eligible owned eater <1H; sets eater to 2H
Reap	Never present as a gameplay CTA

COPY	SLOWS ROT 4x FOR THE NEXT 24H. IT DOES NOT RESET THE CORPSE.

Freshness and Fridge are separate concepts. Fridge slows spoilage 4:1 during its 24H active window. It does not add a fresh 24H corpse life and does not reset the bar.

If a deployed test contract still needs death materialization before power/eat, hide that as compatibility/infrastructure. Canonical contracts should auto-sync logical death inside corpse actions; never force the player to understand Reap.

12 / ACTIONS

ACTION LIBRARY + PHASE GATING

Render only what the current token + phase can legally do. The contract remains final authority.

ACTION	WHEN	SUCCESS RESULT
FEED	Hungry <=12H; also valid rescue state pre-LS	Adds current Meal, cap 36H; clears FAST/Final Bite
FAST	Hungry <=12H pre-LS	FASTING; protection cleared; can survive 0H pre-LS
POISON	Valid attacker + target pre-LS	Normal target half-clock / 1H floor, or Faster -> Final Bite
EAT FRESH	Owned distinct eater Hungry <=12H + Fresh corpse	+12H cap 36H; clears rescue; corpse burns
POWER	Own Fresh corpse pre-LS	Fridge active 24H; spoilage 4:1
EAT ROTTEN	Owned eater <1H + Rotten	Eater ->2H; corpse burns
DEVOUR	Plague onward + Hungry eater + distinct living owned prey	Eater ->36H; prey burns; no corpse
TRUCE	Last Supper + threshold	RETIRE vote; unanimity settles
CLAIM	Settled + winner shares	Pull ETH/WETH prize

LAST SUPPER Feed / Fast / Poison / Power are OFF forever. Fresh / Rotten / Devour / Truce remain.

13 / FEEDBACK

TRANSACTION FEEDBACK

After every confirmed action, show what changed in the game.

ACTION	SUCCESS FEEDBACK
Feed	+[MEAL]H; new clock; Pot change if displayed
Poison	TARGET HIT; clock before -> after; Shield UP 10H; attacker -1H; or Final Bite
Keep Fresh	FRIDGE ACTIVATED; fridge timer; updated ROTS IN
Eat Fresh	+12H to eater; corpse consumed/burned
Eat Rotten	eater ->2H; corpse consumed/burned
Devour	eater ->36H; prey burned
Mint	mint count updated; immediate s_gameStart recheck; LIVE if started

Rule: never leave a giant CONFIRMED button as the final game state. CONFIRMED can appear briefly while the receipt settles, then the UI returns to the action label and shows the game consequence separately.

14 / COMMUNITY

COMMUNITY PRE-MINT + ADMIN

Community mint is a pre-game admission surface. It is not part of the LIVE game navigation.

PLAYER READ/ACTION	BEHAVIOR
getInvitedNftCommunities	Show collection, allocation, max/wallet, active status
s_communityMintprice	Calculate current pre-mint value
s_amountMintPerCollection	Show wallet community usage
preMint(amount, collectionId)	Execute only while pre-mint active and game not started

ADMIN ACTION	BEHAVIOR
addInviteCollection	Add community + contract + caps
allowCommunityMint	Activate community
modifyCollectionMaxAllowed	Increase cap only
modifyCollectionMaxPerWallet	Increase wallet cap only
changeCommunityMintPrice	Update pre-mint price while allowed
endPreMintedPhase	Irreversibly opens public mint

After s_gameStart > 0, /communities must not expose mint execution. Show closed state or redirect. Community tools must never mutate gameplay state.

15 / RESPONSIVE

MOBILE-FIRST / ZERO HORIZONTAL SCROLL

The page width must always equal the device width. Horizontal page scrolling is a bug.

Global CSS requirements:

- html, body: width:100%; max-width:100%; overflow-x:clip.
- All grid/flex children: min-width:0.
- page-shell: width/max-width 100%; padding-inline clamp(12px,2vw,22px).
- img/svg/canvas/video: max-width:100%.
- Never use 100vw inside a padded container.
- No fixed min-width tables that force the whole page sideways.

AREA	MOBILE RULE
Header	Compact mobile navigation; wallet control cannot push viewport.
Leaderboard	Transforms to stacked cards; no 720px horizontal table.
Matrix	Column count adapts to viewport; cell set/order remains fixed.
My Gluttons	Cards/action panels stack cleanly.
Corpse inventory	Freshness/timers stay legible without side scroll.
Long addresses	Wrap/truncate without widening parent.

TEST WIDTHS 320 / 360 / 375 / 390 / 430 / 768 / 1024 / 1440 / 1920.

16 / RPC

STATE SYNC + FAILURE RESILIENCE

A single bad RPC batch must not blank an inventory or make the UI lie.

- Use deployless multicall on ApeChain Curtis when multicall3 is unavailable.
- Small discovery batches; heavy hydration <=20 when combining Inspector + state + metadata.
- On failure retry smaller and preserve already loaded token cards.
- Action receipt -> targeted refresh of attacker/target/eater/corpse/prey IDs only.
- Manual REFRESH LOADED may exist, but actions should not refetch the entire wallet.
- Mint receipt -> immediate Game Start recheck; pre-game polling is fallback only.
- After LIVE latch, stop mint-specific polling permanently.
- If state crosses a clock boundary locally, UI may anticipate death/final-bite display, then canonical reads reconcile.

ERROR UX	Examples: RPC REQUEST FAILED / WRONG DEPLOYMENT OR INSPECTOR WIRING / TARGET PROTECTED / ATTACKER NEEDS >1H / TARGET NEEDS >1H / LAST SUPPER: POISON CLOSED.

17 / TEST CONFIG

MAINNET VS CURTIS

The UI must not encode test-time minutes as game design. All durations shown to users derive from deployed timestamps/state.

CONCERN	RULE
Supply	Read deployment/Starting Population S. Curtis may use 100; mainnet target 2,000.
Compressed time	Test contracts may map hours -> minutes. Frontend displays actual countdown from timestamps; do not hardcode 10 minutes as canonical protection.
Poison protection	Canonical mainnet = 10H. Curtis accelerated equivalent may be 10m if contract uses 1m = 1H.
Clock	Canonical mainnet 24H / 36H. Test deployment can compress.
Freshness	Canonical 24 spoil-hours; test deployment may compress equivalently.
Chain name / currency	Use current deployment config. Do not let test chain labels leak into mainnet build.

18 / MIGRATION

DELETE THE OLD ASSUMPTIONS

The original 49-page manual contains embedded contract code from an intermediate test build. Do not copy these patterns into the final frontend.

OLD / OBSOLETE	FINAL
poisonCooldownUntil / GameEngine__OnCooldown	No attacker cooldown. No frontend state/read/timer.
12H Poison target protection	10H canonical mainnet target protection.
Hardcoded MAX_SUPPLY 100 or 2,000 in UI	Dynamic supply / S.
Manual REGISTER DEATH / REAP button	Hidden infrastructure only; not gameplay.
Corpse card without freshness bar	Always show Freshness + ROTS IN + separate Fridge timer.
Poison Shield OPEN	SHIELD DOWN / SHIELD UP.
Permanent CONFIRMED button	Transient transport status + explicit game consequence.
Matrix before game progress	Progress/phase first; matrix second; leaderboard last.
Horizontal mobile tables	Stacked cards; zero page overflow.
Mint polling forever	Permanent LIVE latch; mint polling stops after start.

19 / ACCEPTANCE

FINAL FRONTEND ACCEPTANCE GATE

Carlos should not call the frontend final until every item below passes on the deployed test contracts.

- ☐ No horizontal page scroll at any target width.
- ☐ No stale Poison cooldown field, copy, timer, check, or ABI-derived UI.
- ☐ Normal target Shield = 10H mainnet rule; DOWN/UP language is correct.
- ☐ Attacker cannot submit Poison with <=1H; target normal <=1H is blocked with human reason.
- ☐ Faster at 0H remains alive pre-LS; Poison produces Final Bite feedback.
- ☐ My Gluttons works on 100-token Curtis deployment and 2,000-token mainnet geometry without code edits.
- ☐ Wallet with NFTs never silently renders empty; RPC errors are explicit and retryable.
- ☐ Mint -> LIVE transition happens immediately after detected Game Start and never reverses.
- ☐ /mint and pre-mint execution are inaccessible after LIVE latch.
- ☐ LIVE hierarchy is progress/phase first, matrix next, leaderboard last.
- ☐ Matrix keeps token positions fixed; Fresh/Rotten remain visible; consumed becomes empty/ash.
- ☐ Leaderboard ranks longest clock; no weakest/vulnerable/Fasting filters or direct Poison buttons.
- ☐ Corpse cards always show Freshness, ROTS IN, and Fridge timer separately.
- ☐ Reap is not exposed as a player mechanic.
- ☐ Every disabled action explains why.
- ☐ Every successful action shows gameplay consequence, not only CONFIRMED.
- ☐ Action receipts target-refresh only affected IDs.
- ☐ No frontend state can make an invalid onchain action appear valid after preflight.
- ☐ Rules page matches Final Frontend Rules v1.1 and Messaging v2.1.
- ☐ Build passes npm install, typecheck/check, production build, and real-device QA.

SHIP RULE	If a frontend behavior conflicts with the deployed ABI or Developer Spec v1.2, stop and resolve the contract/document mismatch. Do not patch gameplay policy in React.

20 / HANDOFF

WHAT CARLOS NEEDS TO DO

This is the implementation sequence for the final pass.

6. 1. Replace any frontend ABI/address bundle with the current deployed contract set and verify Inspector wiring to the same GameEngine + GluttonNFT deployment.
7. 2. Remove all poison cooldown reads/state/copy and implement 10H shield presentation from poisonProtectedUntil.
8. 3. Confirm stage machine uses s_gameStart as permanent LIVE latch and S as live population geometry.
9. 4. Update My Gluttons inventory scanner for dynamic supply, retry ladder, lazy metadata and targeted refresh.
10. 5. Implement final Corpse Inventory: Freshness bar, ROTS IN, separate Fridge timer, no Reap CTA.
11. 6. Implement final Poison target panel and success feedback, with preflight immediately before wallet open.
12. 7. Ensure LIVE order: phase/progress -> matrix -> compact board; keep /leaderboard full route.
13. 8. Apply responsive zero-overflow rules and mobile card transformations.
14. 9. Run contract-by-contract integration tests for Feed, Fast, Poison, Keep Fresh, Eat Fresh, Eat Rotten, Devour, Truce, Claim.
15. 10. Run production build and test at 320 through 1920 widths plus real mobile wallet flow.

FINAL SOURCE OF TRUTH	Use this manual for frontend implementation behavior; use deployed ABIs/contracts + Developer Spec v1.2 for exact callable surfaces. Do not use the old embedded Solidity from the previous Frontend Integration Manual.

22. SMART CONTRACT SOURCE APPENDIX

Carlos source-document reference, reconciled to the current canonical frontend contract surface.

WHY THIS APPENDIX EXISTS: Carlos's original frontend integration document included the Solidity source so the frontend developer could inspect the exact callable surface. This master handoff restores that structure.

AUTHORITY: The deployed contracts and their generated ABI are executable truth. Design Bible v1.3 defines gameplay behavior; Mathematical Lock v1.3 defines constants/formulas; Developer Spec v1.2 defines implementation requirements. This appendix is a frontend integration reference, not a substitute for the deployed repository.

IMPORTANT: The source printed in Carlos's prior PDF was a Curtis accelerated-test snapshot in several places (100 supply and minute-scaled clocks). The frontend MUST NOT hardcode those test constants. Mainnet target is 2,000 and canonical hours; test deployments may intentionally compress time and supply.

CANONICAL DELTAS APPLIED / REQUIRED

- Poison attacker cooldown: NONE. Remove poisonCooldownUntil, GameEngine__OnCooldown, cooldown checks, cooldown writes, frontend reads and cooldown copy.
- Poison normal target protection: 10H mainnet (10 minutes only in an explicitly accelerated 1m=1h Curtis deployment).
- Poison attacker must be alive, non-Fasting, non-Final-Bite and strictly >1H; successful Poison costs attacker exactly 1H.
- Normal Poison target must be alive, non-Final-Bite, unprotected and strictly >1H; new remaining = max(1H, ceil(remaining/2)).
- Poisoning a Faster triggers Final Bite for 1H; a Faster at 0H remains alive before Last Supper.
- Logical death immediately resolves to Fresh. Reap is accounting infrastructure and must not be a user prerequisite for corpse actions.
- Feed / Fast / Poison / Power are permanently OFF after the Last Supper bell. Fresh / Rotten / Live Devour / Truce remain.
- Truce unlock threshold is max(2, ceil(1% of S)), only in LAST_SUPPER. Last Supper trigger is ceil(2.5% of S) OR Day 120, followed by a 1H warning.
- Frontend supply is dynamic: use deployed supply / Starting Population S. Curtis may use 100; mainnet target is 2,000.
- LIVE is a one-way frontend latch once s_gameStart > 0; mint and community mint UI never return in that deployment.

GluttonNFT.sol

SOURCE REFERENCE FROM CARLOS'S ORIGINAL HANDOFF - canonical deltas above control if a deployment differs.

```
// SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 import {ERC721AC} from
 "@limitbreak/creator-token-standards/erc721c/ERC721AC.sol";
 import {ERC2981} from "@openzeppelin/contracts/token/common/ERC2981.sol";
 import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
 import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
 import {Strings} from "@openzeppelin/contracts/utils/Strings.sol";
 import {Base64} from "@openzeppelin/contracts/utils/Base64.sol";
 import { IGameEngine } from "./interfaces/IGameEngine.sol";


 /**
  * @title GluttonNFT
  * @notice ERC721-C ownership and metadata layer for the Gluttons survival
 protocol.
  */
 contract GluttonNFT is ERC721AC, ERC2981, Ownable {
        using Strings for uint256;


        // ========================================
        //               Custom Errors
        // ========================================
        error GluttonNFT__ZeroAddress();
        error GluttonNFT__NotGameEngine();
        error GluttonNFT__TokenDoesNotExist(uint256 tokenId);
        error GluttonNFT__URIsAlreadyFrozen();


        // ========================================
        //               Constants
        // ========================================


        uint96 private constant ROYALTY_BPS = 400; // 4%
        bytes4 private constant ERC4906_INTERFACE_ID = 0x49064906;


        // ========================================
        //               Immutables
        // ========================================
        address public immutable gameEngine;
        address public immutable royaltyTreasury;


        // ========================================
        //                Storage

    // ========================================
    string private s_unrevealedURI;
    string private s_aliveBaseURI;
    string private s_freshBaseURI;
    string private s_rottenBaseURI;


    bool public urisFrozen;


    // ========================================
    //                   Events
    // ========================================
    event URIsFrozen();
    event MetadataUpdate(uint256 _tokenId);
    event BatchMetadataUpdate(uint256 _fromTokenId, uint256 _toTokenId);


    // ========================================
    //                Constructor
    // ========================================
    constructor(address initialOwner_, address gameEngine_, address
royaltyTreasury_)
          ERC721AC("Gluttons", "GLUT")
          Ownable(initialOwner_)
    {
          if (gameEngine_ == address(0) || royaltyTreasury_ == address(0))
revert GluttonNFT__ZeroAddress();


          gameEngine = gameEngine_;
```

```
        royaltyTreasury = royaltyTreasury_;


        // Set 4% ERC-2981 royalty pointing to the RoyaltyTreasury
        _setDefaultRoyalty(royaltyTreasury, ROYALTY_BPS);
    }


    // ========================================
    //            URI Configuration
    // ========================================


    /**
    *
    * @param unrevealedURI_ unreveal URI as json from ardrive
    * @param aliveBaseURI_ alive URI as json from ardrive
    * @param freshBaseURI_ dead fresh base URI, only pass the url with
image, json is onchain.
    * @param rottenBaseURI_ dead rotten base URI, only pass the url with
image, json is onchain.
    */

    function setURIs(
        string memory unrevealedURI_,
        string memory aliveBaseURI_,
        string memory freshBaseURI_,
        string memory rottenBaseURI_
    ) external onlyOwner {
        if (urisFrozen) revert GluttonNFT__URIsAlreadyFrozen();


        s_unrevealedURI = unrevealedURI_;
        s_aliveBaseURI = aliveBaseURI_;
        s_freshBaseURI = freshBaseURI_;
        s_rottenBaseURI = rottenBaseURI_;
    }


    function freezeURIs() external onlyOwner {
        urisFrozen = true;
        emit URIsFrozen();
    }


    // ========================================
    //                   Mint
    // ========================================


    /**
    * @notice Mints a new Glutton.
    * @dev ONLY GameEngine may call this during the pre-game mint phase.
    */
    function gameMint(address to, uint256 amount) external {
        if (msg.sender != gameEngine) revert GluttonNFT__NotGameEngine();
        _mint(to, amount);
    }


    // ========================================
    //                Devour Burn
    // ========================================


    /**
    * @notice Destroys a Glutton consumed by another Glutton (Live Devour
or Corpse Eat).
    * @dev ONLY GameEngine may call this. Normal death does NOT burn the
token.
    */
    function gameBurn(uint256 tokenId) external {
        if (msg.sender != gameEngine) revert GluttonNFT__NotGameEngine();
        _burn(tokenId);
    }

    // ========================================
    //            Metadata & ERC-4906
    // ========================================


    /**
    * @notice Permissionless function to force marketplaces to refresh a
token's image.
    */
    function refreshMetadata(uint256 tokenId) external {
        if (!_exists(tokenId)) revert
GluttonNFT__TokenDoesNotExist(tokenId);
        emit MetadataUpdate(tokenId);
```

```
    }


    /**
    * @notice Derives the exact canonical URI based on GameEngine state.
    */
    function tokenURI(uint256 tokenId) public view virtual override
returns (string memory) {
         if (!_exists(tokenId)) revert
GluttonNFT__TokenDoesNotExist(tokenId);


         uint8 visualState =
IGameEngine(gameEngine).getVisualState(tokenId);


         if (visualState == 0) return s_unrevealedURI;
         if (visualState == 1) return
string(abi.encodePacked(s_aliveBaseURI, tokenId.toString(), ".json"));
         if (visualState == 2) return _onchainURI(tokenId, s_freshBaseURI,
"Dead Fresh");
         if (visualState == 3) return _onchainURI(tokenId, s_rottenBaseURI,
"Dead Rotten");


         // if (visualState == 2) return
string(abi.encodePacked(s_freshBaseURI, tokenId.toString(), ".json"));
         // if (visualState == 3) return
string(abi.encodePacked(s_rottenBaseURI, tokenId.toString(), ".json"));


         return "";
    }


    function _onchainURI(uint256 tokenId, string memory uri, string memory
deadStatus) private pure returns(string memory) {


         bytes memory json   = abi.encodePacked(

             '{',
                   '"name": "Gluttons #', tokenId.toString(), ' (DEAD)",',
                   '"description": "This Glutton starved.",',
                   '"image": "', uri, '",',
                   '"attributes": [',
                         '{',
                               '"trait_type": "Status",',
                               '"value": "', deadStatus,'"',
                         '}',
                   ']',
             '}'
         );
         return string(abi.encodePacked("data:application/json;base64,",
Base64.encode(json)));
    }


    function getTotalMinted() external view returns(uint256) {
         uint256 totalMinted = _totalMinted();
         return totalMinted;
    }


    function getTotalBurned() external view returns(uint256) {
         uint256 totalBurned = _totalBurned();
         return totalBurned;
    }


    // =======================================
    //        ERC165 / ERC2981 / ERC4906
    // =======================================


    function supportsInterface(bytes4 interfaceId) public view
override(ERC721AC, ERC2981) returns (bool) {
         return interfaceId == ERC4906_INTERFACE_ID ||
ERC721AC.supportsInterface(interfaceId)
              || ERC2981.supportsInterface(interfaceId);
    }


    // =======================================
    //         ERC721A Start ID Override
    // =======================================
    function _startTokenId() internal view virtual override returns
(uint256) {
         return 1;
```

```
    }


    // ======================================

        //       Limit Break Ownership Override
        // ======================================
        function _requireCallerIsContractOwner() internal view virtual
override {
            _checkOwner(); // This tells Limit Break to use OpenZeppelin's
Ownable check
        }
}
```

GameEngine.sol

SOURCE REFERENCE FROM CARLOS'S ORIGINAL HANDOFF - canonical deltas above control if a deployment differs.

```
// SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
 import {ReentrancyGuard} from
 "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
 import {IGluttonNFT} from "./interfaces/IGluttonNFT.sol";
 import {IPrizeVault} from "./interfaces/IPrizeVault.sol";
 import {IERC721A} from "../lib/ERC721A/contracts/IERC721A.sol";


 /**
  * @title GameEngine
  * @author Carlos Gutiérrez
  * @notice The deterministic time and state machine for Gluttons.
  */
 contract GameEngine is ReentrancyGuard, Ownable {
        // ========================================
        //              Custom Errors
        // ========================================
        error GameEngine__ZeroAddress();
        error GameEngine__MintClosed();
        error GameEngine__MaxSupplyExceeded();
        error GameEngine__InvalidMintValue();
        error GameEngine__TransferFailed();
        error GameEngine__AlreadyStarted();
        error GameEngine__NotOwner();
        error GameEngine__NotHungry();
        error GameEngine__Dead();

error GameEngine__InvalidValue();
error GameEngine__InvalidState();
error GameEngine__ClockTooLow();

error GameEngine__SelfPoison();
error GameEngine__Protected();
error GameEngine__AlreadySettled();
error GameEngine__NotLastSupper();
error GameEngine__NotUnanimous();
error GameEngine__AlreadyConfigured();
error GameEngine__NoConfigurationToStart();
error GameEngine__AlreadyAllowed();
error GameEngine__NormalMintNotAllowed();
error GameEngine__InvalidMaxPerWalletAmount();
error GameEngine__PreMintPhaseEnded();
error GameEngine__MaxSupplyForInviteExceeded();
error GameEngine__CommunityMintNotAllowed();
error GameEngine__NotCommunityHolder();


// ========================================
//                   Structs
// ========================================
struct TokenState {
     uint64 expiry; // 0 = use global initialExpiry

     uint64 poisonProtectedUntil;
     uint64 finalBiteDeadline;
     uint64 deadAt; // Materialized by logical death/reap
     uint64 spoilCheckpoint;
     uint64 poweredUntil;
     uint32 spoilQ4; // Spoilage tracking
     bool fasting;
     bool deathSettled;
}


struct TruceVote {
     uint256 epoch;
     address ownerAtVote;
}


struct Community {
     string name;
     address collectionAddress;
     uint256 maxTotalAmountAllowed;
     uint256 maxPerWallet;
     bool allowed;

         uint256 amountMinted;
```

```
    }


    // ======================================
    //                Constants
    // ======================================
    uint256 public constant MAX_SUPPLY = 100; // 2000
    uint256 public constant MINT_PRICE = 0.004 ether;
    uint256 public constant FEED_PRICE = 0.0006 ether;
    uint256 public constant MAX_CLOCK_HOURS = 36 minutes; // 36 hours;
    uint256 public constant POISON_PRICE = 0.0004 ether;
    uint256 public constant POWER_PRICE = 0.0003 ether;
    uint32 public constant ROTTEN_THRESHOLD = 5760; // 345600; // 24H *
60m * 60s * 4 units
    uint256 public constant MAX_AMOUNT_PER_WALLET = 4;


    // ======================================
    //                Immutables
    // ======================================
    uint64 public immutable i_startBackstop;


    // ======================================
    //                Variables
    // ======================================


    // Community variables
    uint256 public s_communityMintprice = 0.004 ether;
    uint256 public s_invitedNftIds;
    bool public s_preMintEnd = false;
    mapping(uint256 addressId => Community invitedNft) private
s_invitedNftInfo;
    mapping(address user => mapping(address collection => uint256 amount))
public s_amountMintPerCollection;


    // Normal mint max per wallet record
    mapping(address minter => uint256 mintedAmount) public
s_normalMintAmount;


    address public s_gluttonNFT;
    address public s_prizeVault;
    address public s_pvgTreasury;


    bool public s_isConfigured;


    uint256 public s_totalMinted; // Tracks token IDs

    // Snapshot Variables locked at Game Start
    uint256 public S; // Starting Population
    uint64 public s_gameStart; // The locked timestamp clock origin
    uint64 public s_initialExpiry; // gameStart + 24H


    // Alive/Phase Tracking
    uint256 public s_aliveCount;


    uint256 public s_totalNormalFeeds;
    uint256 public s_completedBars;
    uint256 public s_currentMealSeconds = 24 minutes; // 24 hours; //
Starts at 24H
    uint256 public s_truceEpoch; // Increments on any death or devour to
reset votes


    // Tiebreak & Settlement State
    bool public s_isSettled;
    uint64 public s_lastDeathTimestamp;
    uint256 public s_tiebreakCandidate;


    mapping(uint256 => TokenState) public s_tokenStates;
    mapping(uint256 => TruceVote) public s_truceVotes;
    // Maps winner addresses to their earned shares for the PrizeVault to
query
    mapping(address => uint256) private s_winningShares;


    // ======================================
    //                Events
    // ======================================
```

```
    event Minted(address indexed to, uint256 startTokenId, uint256
quantity);
    event GameStarted(uint64 startTime, uint256 startingPopulation);
    event EndPreMintPhase(uint256 time);


    // ======================================
    //              Constructor
    // ======================================
    constructor(uint64 startBackstop_, address owner_) Ownable(owner_) {
         i_startBackstop = startBackstop_;
    }


    // ======================================
    //          Mint & Initialization
    // ======================================

    /**
     * @notice One-time wiring to connect the GameEngine to the rest of
the protocol.
     */
    function setDependencies(address gluttonNFT_, address prizeVault_,
address pvgTreasury_) external onlyOwner {
          if (s_isConfigured) revert GameEngine__AlreadyConfigured();
          if (gluttonNFT_ == address(0) || prizeVault_ == address(0) ||
pvgTreasury_ == address(0)) {
              revert GameEngine__ZeroAddress();
          }


          s_gluttonNFT = gluttonNFT_;
          s_prizeVault = prizeVault_;
          s_pvgTreasury = pvgTreasury_;


          s_isConfigured = true;
    }


    /**
     * @notice Mints Gluttons pre-game. Routes funds 95% Pot / 5% PVG.
     */
    function mint(uint256 amount) external payable nonReentrant {
          if (!s_preMintEnd) revert GameEngine__NormalMintNotAllowed();


          if (s_normalMintAmount[msg.sender] + amount >
MAX_AMOUNT_PER_WALLET) {
              revert GameEngine__InvalidMaxPerWalletAmount();
          }


          if (!s_isConfigured) revert GameEngine__NoConfigurationToStart();
          if (s_gameStart != 0 || block.timestamp >= i_startBackstop) revert
GameEngine__MintClosed();
          if (s_totalMinted + amount > MAX_SUPPLY) revert
GameEngine__MaxSupplyExceeded();
          if (msg.value != amount * MINT_PRICE) revert
GameEngine__InvalidMintValue();


          uint256 startTokenId = s_totalMinted + 1;


          // 95% to PrizeVault, 5% to PVGTreasury
          uint256 pvgShare = (msg.value * 5) / 100;
          uint256 potShare = msg.value - pvgShare;


          (bool pvgSuccess,) = s_pvgTreasury.call{value: pvgShare}("");
          if (!pvgSuccess) revert GameEngine__TransferFailed();

          (bool potSuccess,) = s_prizeVault.call{value: potShare}("");
          if (!potSuccess) revert GameEngine__TransferFailed();


          s_totalMinted += amount;
          s_aliveCount += amount;


          s_normalMintAmount[msg.sender] += amount;


          IGluttonNFT(s_gluttonNFT).gameMint(msg.sender, amount);


          emit Minted(msg.sender, startTokenId, amount);
```

```
        // Auto-start if sold out
        if (s_totalMinted == MAX_SUPPLY) {
            _startGame(uint64(block.timestamp));
        }
    }


    /**
     * @notice Pre-Mints Gluttons pre-game. Only allowed if the collection
to use is registerd.
     */
    function preMint(uint256 amount, uint256 collectionId) external
payable nonReentrant {
        if (s_preMintEnd) revert GameEngine__PreMintPhaseEnded();


        if (!s_isConfigured) revert GameEngine__NoConfigurationToStart();
        if (s_gameStart != 0 || block.timestamp >= i_startBackstop) revert
GameEngine__MintClosed();


        Community memory collection = s_invitedNftInfo[collectionId];


        if (!collection.allowed) revert
GameEngine__CommunityMintNotAllowed();


        if (IERC721A(collection.collectionAddress).balanceOf(msg.sender)
== 0) revert GameEngine__NotCommunityHolder();


        if (collection.amountMinted + amount >
collection.maxTotalAmountAllowed) {
            revert GameEngine__MaxSupplyForInviteExceeded();
        }
        if
(s_amountMintPerCollection[msg.sender][collection.collectionAddress] +
amount > collection.maxPerWallet) {

            revert GameEngine__InvalidMaxPerWalletAmount();
        }
        if (s_totalMinted + amount > MAX_SUPPLY) revert
GameEngine__MaxSupplyExceeded();
        if (msg.value != amount * s_communityMintprice) revert
GameEngine__InvalidMintValue();


        uint256 startTokenId = s_totalMinted + 1;


        // 95% to PrizeVault, 5% to PVGTreasury
        uint256 pvgShare = (msg.value * 5) / 100;
        uint256 potShare = msg.value - pvgShare;


        (bool pvgSuccess,) = s_pvgTreasury.call{value: pvgShare}("");
        if (!pvgSuccess) revert GameEngine__TransferFailed();


        (bool potSuccess,) = s_prizeVault.call{value: potShare}("");
        if (!potSuccess) revert GameEngine__TransferFailed();


        s_totalMinted += amount;
        s_aliveCount += amount;



s_amountMintPerCollection[msg.sender][collection.collectionAddress] +=
amount;
        s_invitedNftInfo[collectionId].amountMinted += amount;


        IGluttonNFT(s_gluttonNFT).gameMint(msg.sender, amount);


        emit Minted(msg.sender, startTokenId, amount);


        // Auto-start if sold out
        if (s_totalMinted == MAX_SUPPLY) {
            _startGame(uint64(block.timestamp));
        }
    }


    /**
```

```
    * @notice Permissionless initialization if mint doesn't sell out by
backstop.
    */
    function ensureStarted() public {
          if (s_gameStart != 0) revert GameEngine__AlreadyStarted();
          if (block.timestamp >= i_startBackstop) {
              // Late initialization uses the published backstop, preventing
clock manipulation

              _startGame(i_startBackstop);
          }
    }


    function _startGame(uint64 _startTime) private {
          s_gameStart = _startTime;
          s_initialExpiry = _startTime + 24 minutes; // hours
          S = s_totalMinted; // Snapshot starting population
          emit GameStarted(_startTime, S);
    }


    // ======================================
    //            Time & State Helpers
    // ======================================


    /**
    * @dev Resolves the lazy expiry load.
    */
    function effectiveExpiry(uint256 tokenId) public view returns (uint64)
{
          uint64 stored = s_tokenStates[tokenId].expiry;
          return stored == 0 ? s_initialExpiry : stored;
    }


    /**
    * @dev Checks if the game has entered the Last Supper phase.
    */
    function _isLastSupper() internal view returns (bool) {
          if (S == 0) return false;
          uint256 maxTrucePop = (S + 99) / 100; // ceil(0.01 * S)
          if (maxTrucePop < 2) maxTrucePop = 2;
          return s_aliveCount <= maxTrucePop;
    }


    /**
    * @dev The canonical death evaluator. Enforces FASTING at 0H survival
and Final Bite lock.
    */
    function _checkTokenDeath(TokenState memory ts, uint64 exp) internal
view returns (bool isDead, uint64 actualDeadAt) {
          if (ts.deadAt > 0) {
              return (true, ts.deadAt);
          }
          if (ts.finalBiteDeadline > 0) {
              if (block.timestamp >= ts.finalBiteDeadline) {
                  return (true, ts.finalBiteDeadline);

              }
          } else if (block.timestamp >= exp) {
              if (ts.fasting) {
                  if (_isLastSupper()) {
                        return (true, uint64(block.timestamp)); // Killed by
the Last Supper bell
                  }
              } else {
                  return (true, exp);
              }
          }
          return (false, 0);
    }


    /**
    * @notice The authoritative visual state derived for GluttonNFT.
    */
    function getVisualState(uint256 tokenId) external view returns (uint8)
{
          if (s_gameStart == 0) return 0; // UNREVEALED


          TokenState memory ts = s_tokenStates[tokenId];
          uint64 exp = effectiveExpiry(tokenId);


          // Canonical logical death check using the new helper
```

```
          (bool isLogicallyDead, uint64 actualDeadAt) = _checkTokenDeath(ts,
exp);


          if (!isLogicallyDead) return 1; // ALIVE


          // Spoilage calculation
          uint256 virtualSpoil = _virtualSpoilQ4(ts, actualDeadAt);


          if (virtualSpoil < ROTTEN_THRESHOLD) {
              return 2; // FRESH CORPSE
          }


          return 3; // ROTTEN
    }


    function _virtualSpoilQ4(TokenState memory ts, uint64 actualDeadAt)
internal view returns (uint256) {
          uint256 spoil = ts.spoilQ4;


          if (spoil >= ROTTEN_THRESHOLD) {
              return spoil;

         }


         uint64 lastCheck = ts.spoilCheckpoint > 0 ? ts.spoilCheckpoint :
actualDeadAt;


         if (block.timestamp <= lastCheck) {
             return spoil;
         }


         uint256 elapsed = block.timestamp - lastCheck;


         if (ts.poweredUntil > lastCheck) {
             if (block.timestamp <= ts.poweredUntil) {
                    // Entire period refrigerated:
                    // 4 real seconds = 1 normal spoil second
                    spoil += elapsed;
             } else {
                    // Part refrigerated, part normal
                    uint256 poweredElapsed = ts.poweredUntil - lastCheck;


                    uint256 unpoweredElapsed = block.timestamp -
ts.poweredUntil;


                    spoil += poweredElapsed + (unpoweredElapsed * 4);
             }
         } else {
             // Normal spoilage
             spoil += elapsed * 4;
         }
         return spoil;
    }


    function _clearRescueState(uint256 tokenId) internal {
         s_tokenStates[tokenId].fasting = false;
         s_tokenStates[tokenId].finalBiteDeadline = 0;
    }


    // ======================================
    //              Feed & Metabolism
    // ======================================


    function feed(uint256 tokenId) external payable nonReentrant {
         if (msg.sender != IGluttonNFT(s_gluttonNFT).ownerOf(tokenId))
revert GameEngine__NotOwner();


          TokenState storage ts = s_tokenStates[tokenId];

         uint64 exp = effectiveExpiry(tokenId);


         // Use the new helper to ensure they aren't actually dead
```

```
        (bool isDead, ) = _checkTokenDeath(ts, exp);
        if (isDead) revert GameEngine__Dead();


        // Must be Hungry (<= 12M remaining) OR in a rescue state
        bool isHungry = (exp > block.timestamp && exp - block.timestamp <=
12 minutes);
        bool needsRescue = ts.fasting || ts.finalBiteDeadline > 0;


        if (!isHungry && !needsRescue) revert GameEngine__NotHungry();


        // 2. Payment Routing
        if (msg.value != FEED_PRICE) revert GameEngine__InvalidValue();
        uint256 pvgShare = (msg.value * 8) / 100;
        uint256 potShare = msg.value - pvgShare;


        (bool pvgSuccess,) = s_pvgTreasury.call{value: pvgShare}("");
        if (!pvgSuccess) revert GameEngine__TransferFailed();


        (bool potSuccess,) = s_prizeVault.call{value: potShare}("");
        if (!potSuccess) revert GameEngine__TransferFailed();


        // 3. Apply Time
        uint64 newBase = uint64(block.timestamp > exp ? block.timestamp :
exp);
        uint64 maxAllowed = uint64(block.timestamp + MAX_CLOCK_HOURS);


        uint64 projectedExpiry = newBase + uint64(s_currentMealSeconds);
        ts.expiry = projectedExpiry > maxAllowed ? maxAllowed :
projectedExpiry;


        // 4. Rescue Reset
        _clearRescueState(tokenId);


        // 5. Metabolism Advance
        s_totalNormalFeeds++;


        if (s_totalNormalFeeds % S == 0) {
            s_completedBars++;
            uint256 nextMeal = (s_currentMealSeconds * 9800) / 10000;
            if ((s_currentMealSeconds * 9800) % 10000 != 0) nextMeal++;
            if (nextMeal < 120) nextMeal = 120; // Testnet floor
            s_currentMealSeconds = nextMeal;
        }

    }


    // ========================================
    //           Fasting & Poisoning
    // ========================================


    function enterFast(uint256 tokenId) external nonReentrant {
         if (msg.sender != IGluttonNFT(s_gluttonNFT).ownerOf(tokenId))
revert GameEngine__NotOwner();


         TokenState storage ts = s_tokenStates[tokenId];
         uint64 exp = effectiveExpiry(tokenId);


         // Use the new helper
         (bool isDead, ) = _checkTokenDeath(ts, exp);
         if (isDead) revert GameEngine__Dead();


         // Must be hungry (<= 12M)
         if (exp > block.timestamp && exp - block.timestamp > 12 minutes)
revert GameEngine__NotHungry();


         ts.fasting = true;
         ts.poisonProtectedUntil = 0; // Cancels target protection
    }


    function poison(uint256 attackerId, uint256 targetId) external payable
nonReentrant {
         if (msg.value != POISON_PRICE) revert GameEngine__InvalidValue();
```

```
        if (attackerId == targetId) revert GameEngine__SelfPoison();


        // 1. Validate Attacker
        if (msg.sender != IGluttonNFT(s_gluttonNFT).ownerOf(attackerId))
revert GameEngine__NotOwner();
        TokenState storage attackerTs = s_tokenStates[attackerId];
        uint64 attackerExp = effectiveExpiry(attackerId);


        // Helper check for attacker
        (bool attackerDead, ) = _checkTokenDeath(attackerTs, attackerExp);
        if (attackerDead) revert GameEngine__Dead();


        if (attackerTs.fasting || attackerTs.finalBiteDeadline > 0) revert
GameEngine__InvalidState();
        if (attackerExp - block.timestamp <= 1 minutes) revert
GameEngine__ClockTooLow();

      // 2. Validate Target
      TokenState storage targetTs = s_tokenStates[targetId];
      uint64 targetExp = effectiveExpiry(targetId);


      // Helper check for target (Allows Fasters at 0H to be targeted)
      (bool targetDead, ) = _checkTokenDeath(targetTs, targetExp);
      if (targetDead) revert GameEngine__Dead();


      if (targetTs.finalBiteDeadline > 0) revert
GameEngine__InvalidState(); // Already in Final Bite


      // 3. Apply Target Logic
      if (targetTs.fasting) {
          targetTs.finalBiteDeadline = uint64(block.timestamp + 1
minutes);
      } else {
          if (targetExp - block.timestamp <= 1 minutes) revert
GameEngine__ClockTooLow();
          if (block.timestamp < targetTs.poisonProtectedUntil) revert
GameEngine__Protected();


          uint64 remaining = targetExp - uint64(block.timestamp);
          uint64 halfRemaining = (remaining + 1) / 2;
          uint64 newRemaining = halfRemaining > 1 minutes ?
halfRemaining : 1 minutes;


          targetTs.expiry = uint64(block.timestamp) + newRemaining;
          targetTs.poisonProtectedUntil = uint64(block.timestamp + 10 minutes); // 10H canonical; 10m in accelerated test
      }


      // 4. Apply Attacker Penalty
      attackerTs.expiry = attackerExp - 1 minutes;

      // 5. Route Funds
      uint256 pvgShare = (msg.value * 8) / 100;
      uint256 potShare = msg.value - pvgShare;


      (bool pvgSuccess,) = s_pvgTreasury.call{value: pvgShare}("");
      if (!pvgSuccess) revert GameEngine__TransferFailed();


      (bool potSuccess,) = s_prizeVault.call{value: potShare}("");
      if (!potSuccess) revert GameEngine__TransferFailed();

  }


  // ========================================
  //               Spoilage & Power
  // ========================================


  /**
  * @dev Syncs virtual spoilage up to the current block.
  */
  function _syncSpoilage(uint256 tokenId) internal {
        TokenState storage ts = s_tokenStates[tokenId];
        if (ts.deadAt == 0) return; // Not dead
        if (ts.spoilQ4 >= ROTTEN_THRESHOLD) return; // Already rotten


        uint64 lastCheck = ts.spoilCheckpoint > 0 ? ts.spoilCheckpoint :
```

```
ts.deadAt;
          if (block.timestamp <= lastCheck) return;


          uint256 elapsed = block.timestamp - lastCheck;


          if (ts.poweredUntil > lastCheck) {
              if (block.timestamp <= ts.poweredUntil) {
                    // Fully powered
                    ts.spoilQ4 += uint32(elapsed * 1);
              } else {
                    // Partially powered
                    uint256 poweredElapsed = ts.poweredUntil - lastCheck;
                    uint256 unpoweredElapsed = block.timestamp -
ts.poweredUntil;
                    ts.spoilQ4 += uint32((poweredElapsed * 1) +
(unpoweredElapsed * 4));
              }
          } else {
              // Unpowered
              ts.spoilQ4 += uint32(elapsed * 4);
          }


          ts.spoilCheckpoint = uint64(block.timestamp);
    }


    function powerFridge(uint256 tokenId) external payable nonReentrant {
          if (msg.value != POWER_PRICE) revert GameEngine__InvalidValue();
          if (msg.sender != IGluttonNFT(s_gluttonNFT).ownerOf(tokenId))
revert GameEngine__NotOwner();


          // 1. Ensure token is a Fresh Corpse

         _syncSpoilage(tokenId);
         TokenState storage ts = s_tokenStates[tokenId];
         if (ts.deadAt == 0 || ts.spoilQ4 >= ROTTEN_THRESHOLD) revert
GameEngine__InvalidState();
         if (ts.poweredUntil >= block.timestamp) revert
GameEngine__InvalidState(); // No stacking


         // 2. Apply Power
         ts.poweredUntil = uint64(block.timestamp + 24 minutes);


         // 3. Route Funds (92% / 8%)
         uint256 pvgShare = (msg.value * 8) / 100;
         uint256 potShare = msg.value - pvgShare;


         (bool pvgSuccess,) = s_pvgTreasury.call{value: pvgShare}("");
         if (!pvgSuccess) revert GameEngine__TransferFailed();


         (bool potSuccess,) = s_prizeVault.call{value: potShare}("");
         if (!potSuccess) revert GameEngine__TransferFailed();
    }


    // ======================================
    //              Consumption
    // ======================================


    function liveDevour(uint256 eaterId, uint256 preyId) external
nonReentrant {
         if (eaterId == preyId) revert GameEngine__InvalidState();


         IGluttonNFT nft = IGluttonNFT(s_gluttonNFT);
         if (msg.sender != nft.ownerOf(eaterId) || msg.sender !=
nft.ownerOf(preyId)) revert GameEngine__NotOwner();


         if (s_aliveCount > (S * 50) / 100 && s_currentMealSeconds > 120)
revert GameEngine__InvalidState();


         TokenState storage eaterTs = s_tokenStates[eaterId];
         TokenState storage preyTs = s_tokenStates[preyId];


         uint64 eaterExp = effectiveExpiry(eaterId);
         uint64 preyExp = effectiveExpiry(preyId);
```

```
        // Use helper for both
        (bool eaterDead, ) = _checkTokenDeath(eaterTs, eaterExp);
        (bool preyDead, ) = _checkTokenDeath(preyTs, preyExp);
        if (eaterDead || preyDead) revert GameEngine__Dead();

        if (eaterExp - block.timestamp > 12 minutes) revert
GameEngine__NotHungry();


        eaterTs.expiry = uint64(block.timestamp + MAX_CLOCK_HOURS);
        _clearRescueState(eaterId);


        preyTs.deadAt = uint64(block.timestamp);
        s_aliveCount--;
        s_truceEpoch++;


        nft.gameBurn(preyId);
    }


    // ========================================
    //               Eat Corpse
    // ========================================


    function consumeCorpse(uint256 eaterId, uint256 corpseId) external
nonReentrant {
        if (s_isSettled) revert GameEngine__AlreadySettled();
        if (eaterId == corpseId) revert GameEngine__InvalidState();


        IGluttonNFT nft = IGluttonNFT(s_gluttonNFT);
        if (msg.sender != nft.ownerOf(eaterId) || msg.sender !=
nft.ownerOf(corpseId)) revert GameEngine__NotOwner();


        TokenState storage eaterTs = s_tokenStates[eaterId];
        uint64 eaterExp = effectiveExpiry(eaterId);


        // Helper check for the eater
        (bool eaterDead, ) = _checkTokenDeath(eaterTs, eaterExp);
        if (eaterDead) revert GameEngine__Dead();


        TokenState storage corpseTs = s_tokenStates[corpseId];
        if (corpseTs.deadAt == 0) revert GameEngine__InvalidState(); //
Must be dead


        _syncSpoilage(corpseId);


        if (corpseTs.spoilQ4 < ROTTEN_THRESHOLD) {
             if (eaterExp > block.timestamp && eaterExp - block.timestamp >
12 minutes) revert GameEngine__NotHungry();


              uint64 newBase = uint64(block.timestamp > eaterExp ?
block.timestamp : eaterExp);

             uint64 maxAllowed = uint64(block.timestamp + MAX_CLOCK_HOURS);
             uint64 projectedExpiry = newBase + 12 minutes;


             eaterTs.expiry = projectedExpiry > maxAllowed ? maxAllowed :
projectedExpiry;
         } else {
             if (eaterExp > block.timestamp && eaterExp - block.timestamp
>= 1 minutes) revert GameEngine__NotHungry();
             eaterTs.expiry = uint64(block.timestamp + 2 minutes);
         }


         _clearRescueState(eaterId);
         nft.gameBurn(corpseId);
    }


    // ========================================
    //           Reap & Tiebreak Logic
    // ========================================


    /**
    * @notice Permissionless function to materialize deaths and update
```

```
tiebreaks.
    */
    function reap(uint256[] calldata tokenIds) external nonReentrant {
          if (s_isSettled) return;


          for (uint256 i = 0; i < tokenIds.length; i++) {
              uint256 tid = tokenIds[i];
              TokenState storage ts = s_tokenStates[tid];
              if (ts.deathSettled) continue;


              uint64 exp = effectiveExpiry(tid);


              // Use the single truth helper
              (bool isDead, uint64 actualDeadAt) = _checkTokenDeath(ts,
exp);


              if (isDead) {
                     ts.deadAt = actualDeadAt;
                     ts.deathSettled = true;
                     s_aliveCount--;
                     s_truceEpoch++;


                     if (actualDeadAt > s_lastDeathTimestamp) {
                         s_lastDeathTimestamp = actualDeadAt;
                         s_tiebreakCandidate = tid;

                 } else if (actualDeadAt == s_lastDeathTimestamp) {
                      if (s_tiebreakCandidate == 0 || tid <
s_tiebreakCandidate) {
                           s_tiebreakCandidate = tid;
                      }
                 }
              }
          }


          if (s_aliveCount == 0 && s_tiebreakCandidate != 0) {
              s_isSettled = true;
              address winner =
IGluttonNFT(s_gluttonNFT).ownerOf(s_tiebreakCandidate);
              s_winningShares[winner] = 1;
              IPrizeVault(s_prizeVault).finalize(1);
          }
    }


    // ======================================
    //                Endgame Truce
    // ======================================


    /**
    * @notice Allows a surviving player to register their vote for a
Truce.
    */
    function voteTruce(uint256 tokenId) external {
          if (s_isSettled) revert GameEngine__AlreadySettled();
          if (msg.sender != IGluttonNFT(s_gluttonNFT).ownerOf(tokenId))
revert GameEngine__NotOwner();


          // Ensure LAST_SUPPER TRUCE population constraints
          uint256 maxTrucePop = (S + 99) / 100; // ceil(0.01 * S)
          if (maxTrucePop < 2) maxTrucePop = 2;
          if (s_aliveCount > maxTrucePop) revert
GameEngine__NotLastSupper();


          s_truceVotes[tokenId] = TruceVote({epoch: s_truceEpoch,
ownerAtVote: msg.sender});
    }


    /**
    * @notice Settles the game if a Truce is unanimous, or if only 1
Glutton remains.
    */

    function settleGame(uint256[] calldata liveTokenIds) external
nonReentrant {
         if (s_isSettled) revert GameEngine__AlreadySettled();
         if (liveTokenIds.length != s_aliveCount) revert
GameEngine__InvalidState();
```

```
        if (s_aliveCount == 1) {
            // Single Survivor Win
            s_isSettled = true;
            address winner =
IGluttonNFT(s_gluttonNFT).ownerOf(liveTokenIds[0]);
            s_winningShares[winner] = 1;
            IPrizeVault(s_prizeVault).finalize(1);
            return;
        }


        // Truce Verification
        IGluttonNFT nft = IGluttonNFT(s_gluttonNFT);


        for (uint256 i = 0; i < liveTokenIds.length; i++) {
            uint256 tid = liveTokenIds[i];
            TruceVote memory vote = s_truceVotes[tid];


            // Vote must be from the current epoch and the token must not
have changed hands
            if (vote.epoch != s_truceEpoch || vote.ownerAtVote !=
nft.ownerOf(tid)) {
                revert GameEngine__NotUnanimous();
            }
        }


        // If loop completes, Truce is unanimous
        s_isSettled = true;


        // Snapshot entitlements
        for (uint256 i = 0; i < liveTokenIds.length; i++) {
            address owner = nft.ownerOf(liveTokenIds[i]);
            s_winningShares[owner]++;
        }


        IPrizeVault(s_prizeVault).finalize(s_aliveCount);
    }


    // ========================================
    //       PrizeVault Integration (View)
    // ========================================

    /**
     * @notice Called by PrizeVault to determine how many shares an
address won.
     */
    function getWinningShares(address account) external view returns
(uint256) {
         return s_winningShares[account];
    }


    /**
     * @notice Called by RoyaltyTreasury to check routing destination.
     */
    function isSettled() external view returns (bool) {
          return s_isSettled;
    }


    // Community functions


    /**
     * @dev function to add and nft collection for mint
     * @param name_ name of collection
     * @param collection_ nft collection address
     * @param maxAllowed_ max allowed to mint per collection
     * @param maxPerWallet_ max per wallet on invite nft collection
     */
    function addInviteCollection(string memory name_, address collection_,
uint256 maxAllowed_, uint256 maxPerWallet_)
          external
          onlyOwner
    {
          if (bytes(name_).length == 0) revert GameEngine__InvalidValue();
          if (collection_ == address(0)) revert GameEngine__ZeroAddress();
          if (maxAllowed_ == 0) revert GameEngine__InvalidValue();
          if (maxPerWallet_ == 0) revert GameEngine__InvalidValue();
          uint256 invitedNftIds = s_invitedNftIds;
          s_invitedNftInfo[invitedNftIds] = Community({
                name: name_,
```

```
                collectionAddress: collection_,
                maxTotalAmountAllowed: maxAllowed_,
                maxPerWallet: maxPerWallet_,
                allowed: false,
                amountMinted: 0
            });
            s_invitedNftIds++;
    }

    /**
    * @dev Allow mint for a collection
    * @param collectionId collection Id to enter info and allow the mint
    */
    function allowCommunityMint(uint256 collectionId) external onlyOwner {
          if (collectionId >= s_invitedNftIds) revert
GameEngine__InvalidValue();
          Community storage collection = s_invitedNftInfo[collectionId];
          if (collection.allowed == true) revert
GameEngine__AlreadyAllowed();
          collection.allowed = true;
    }


    /**
    * @dev Modify the collection max allowed mint
    * @param collectionId collection Id
    * @param newAmount new amount
    */
    function modifyCollectionMaxAllowed(uint256 collectionId, uint256
newAmount) external onlyOwner {
          if (collectionId >= s_invitedNftIds) revert
GameEngine__InvalidValue();
          Community storage collection = s_invitedNftInfo[collectionId];
          if (newAmount <= collection.maxTotalAmountAllowed) revert
GameEngine__InvalidValue();
          collection.maxTotalAmountAllowed = newAmount;
    }


    /**
    * @dev Modify the collection max per wallet allowed mint
    * @param collectionId collection Id
    * @param newAmount new amount
    */
    function modifyCollectionMaxPerWallet(uint256 collectionId, uint256
newAmount) external onlyOwner {
          if (collectionId >= s_invitedNftIds) revert
GameEngine__InvalidValue();
          Community storage collection = s_invitedNftInfo[collectionId];
          if (newAmount <= collection.maxPerWallet) revert
GameEngine__InvalidValue();
          collection.maxPerWallet = newAmount;
    }


    /**
    * @dev End Pre-Mint phase and allow normal mint

        */
        function endPreMintedPhase() external onlyOwner {
              s_preMintEnd = true;
              emit EndPreMintPhase(block.timestamp);
        }


        /**
        * @dev Change the mint price of communities pre-mint sale
        * @param newPrice new Price
        */
        function changeCommunityMintPrice(uint256 newPrice) external onlyOwner
 {
              s_communityMintprice = newPrice;
        }


        /**
        * @dev View function to get the invited communities and get data for
 owner to modify with funtions or for front end
        */
        function getInvitedNftCommunities() public view returns (Community[]
 memory) {
              uint256 ids = s_invitedNftIds;
              Community[] memory nftCommunities = new Community[](ids);
              for (uint256 i = 0; i < ids; i++) {
                  nftCommunities[i] = s_invitedNftInfo[i];
              }
              return nftCommunities;
        }
 }
```

Inspector.sol

SOURCE REFERENCE FROM CARLOS'S ORIGINAL HANDOFF - canonical deltas above control if a deployment differs.

```
// NOTE: current canonical frontend should prefer a phase value that distinguishes LS_WARNING and LAST_SUPPER and includes
Day-120 timing.
// SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 import { IGluttonNFT } from "./interfaces/IGluttonNFT.sol";
 import { IGameEngine } from "./interfaces/IGameEngine.sol";


 /**

 * @title Inspector
 * @author Carlos Gutiérrez
 * @notice Read-only aggregate for frontend display.
 *
 * Must do the following:
 * - Role: Exposes the canonical state before a user makes a purchase.
 * - Reads Only: It must have absolutely no state-changing functions (no
writes).
 * - No Authority: The frontend must never treat the Inspector as a source
of truth for executing transactions; it is purely for display and
aggregation.
 *
 */
contract Inspector {


      IGameEngine private immutable i_gameEngine;
      IGluttonNFT private immutable i_gluttonNFT;


      struct TokenStateView {
          address owner;
          uint8 visualState; // 0=UNREVEALED, 1=ALIVE, 2=FRESH, 3=ROTTEN
          uint64 expiry;
          bool isHungry;
      }


      struct GlobalState {
          uint256 aliveCount; // s_aliveCount()
          uint256 currentMealSeconds; // s_currentMealSeconds()
          bool isSettled; // isSettled()
          string currentPhase; // e.g., "FEAST", "PLAGUE", "LAST_SUPPER"
      }


      constructor(address gameEngine_, address gluttonNFT_) {
          i_gameEngine = IGameEngine(gameEngine_);
          i_gluttonNFT = IGluttonNFT(gluttonNFT_);
      }


      function getTokenView(uint256 tokenId) external view returns
(TokenStateView memory) {
          uint64 exp = i_gameEngine.effectiveExpiry(tokenId);
          uint8 vState = i_gameEngine.getVisualState(tokenId);


          bool hungry = false;


          // Only an ALIVE glutton can be hungry
          if (vState == 1) {

               if (exp <= block.timestamp) {
                   hungry = true; // It's a Faster at 0H or in Final Bite
               } else if (exp - block.timestamp <= 12 minutes) {
                   hungry = true; // Normal hunger threshold
               }
          }


          return TokenStateView({
               owner: i_gluttonNFT.ownerOf(tokenId),
               visualState: vState,
               expiry: exp,
               isHungry: hungry
          });
    }
```

```
    function getGlobalView() external view returns(GlobalState memory) {
         uint256 alive = i_gameEngine.s_aliveCount();
         uint256 meal = i_gameEngine.s_currentMealSeconds();
         bool settled = i_gameEngine.isSettled();
         uint256 S = i_gameEngine.S(); // Assuming GameEngine has an S()
getter


         string memory phase = "FEAST";


         if (settled) {
               phase = "SETTLED";
         } else if (alive > 0 && S > 0) {
               // Plague triggers at <= 50% population OR 2H meal floor
               if (alive <= (S * 50) / 100 || meal <= 120) {
                   phase = "PLAGUE";
               }
               // Last Supper triggers at <= 2.5% population (simplified for
view)
               if (alive <= (S * 25) / 1000) {
                   phase = "LAST_SUPPER";
               }
         } else if (i_gameEngine.s_gameStart() == 0) {
               phase = "PRE_GAME";
         }


         return GlobalState({
               aliveCount: alive,
               currentMealSeconds: meal,
               isSettled: settled,
               currentPhase: phase
         });

        }


 }
```

PrizeVault.sol

SOURCE REFERENCE FROM CARLOS'S ORIGINAL HANDOFF - canonical deltas above control if a deployment differs.

```
// SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 import { IGluttonNFT } from "./interfaces/IGluttonNFT.sol";
 import { IGameEngine } from "./interfaces/IGameEngine.sol";


 /**

 * @title Inspector
 * @author Carlos Gutiérrez
 * @notice Read-only aggregate for frontend display.
 *
 * Must do the following:
 * - Role: Exposes the canonical state before a user makes a purchase.
 * - Reads Only: It must have absolutely no state-changing functions (no
writes).
 * - No Authority: The frontend must never treat the Inspector as a source
of truth for executing transactions; it is purely for display and
aggregation.
 *
 */
contract Inspector {


      IGameEngine private immutable i_gameEngine;
      IGluttonNFT private immutable i_gluttonNFT;


      struct TokenStateView {
          address owner;
          uint8 visualState; // 0=UNREVEALED, 1=ALIVE, 2=FRESH, 3=ROTTEN
          uint64 expiry;
          bool isHungry;
      }


      struct GlobalState {
          uint256 aliveCount; // s_aliveCount()
          uint256 currentMealSeconds; // s_currentMealSeconds()
          bool isSettled; // isSettled()
          string currentPhase; // e.g., "FEAST", "PLAGUE", "LAST_SUPPER"
      }


      constructor(address gameEngine_, address gluttonNFT_) {
          i_gameEngine = IGameEngine(gameEngine_);
          i_gluttonNFT = IGluttonNFT(gluttonNFT_);
      }


      function getTokenView(uint256 tokenId) external view returns
(TokenStateView memory) {
          uint64 exp = i_gameEngine.effectiveExpiry(tokenId);
          uint8 vState = i_gameEngine.getVisualState(tokenId);


          bool hungry = false;


          // Only an ALIVE glutton can be hungry
          if (vState == 1) {

               if (exp <= block.timestamp) {
                   hungry = true; // It's a Faster at 0H or in Final Bite
               } else if (exp - block.timestamp <= 12 minutes) {
                   hungry = true; // Normal hunger threshold
               }
         }


         return TokenStateView({
               owner: i_gluttonNFT.ownerOf(tokenId),
               visualState: vState,
               expiry: exp,
               isHungry: hungry
         });
    }


    function getGlobalView() external view returns(GlobalState memory) {
```

```
        uint256 alive = i_gameEngine.s_aliveCount();
        uint256 meal = i_gameEngine.s_currentMealSeconds();
        bool settled = i_gameEngine.isSettled();
        uint256 S = i_gameEngine.S(); // Assuming GameEngine has an S()
getter


        string memory phase = "FEAST";


        if (settled) {
              phase = "SETTLED";
        } else if (alive > 0 && S > 0) {
              // Plague triggers at <= 50% population OR 2H meal floor
              if (alive <= (S * 50) / 100 || meal <= 120) {
                  phase = "PLAGUE";
              }
              // Last Supper triggers at <= 2.5% population (simplified for
view)
              if (alive <= (S * 25) / 1000) {
                  phase = "LAST_SUPPER";
              }
        } else if (i_gameEngine.s_gameStart() == 0) {
              phase = "PRE_GAME";
        }


        return GlobalState({
              aliveCount: alive,
              currentMealSeconds: meal,
              isSettled: settled,
              currentPhase: phase
        });

       }


 }



PrizeVault.sol

 // SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
 import {SafeERC20} from
 "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
 import {ReentrancyGuard} from
 "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
 import { IGameEngine } from "./interfaces/IGameEngine.sol";


 /**
  * @title PrizeVault
  * @author Carlos Gutiérrez
  * @notice Holds the active-game ETH and WETH prize pool.
  *
  * RESPONSIBILITIES
  * ----------------
  *
  * - Must accept incoming ETH.
  * - Must implement a pull-based claim system for winners to withdraw
 their share after the game is settled.
  * - Must distribute both ETH and WETH balances in-kind to the winners.
  * - Strict Security: The contract owner/admin must never be able to
 withdraw from this vault during the active game.
  *
  *
  * SECURITY MODEL
  * --------------
  *
  * There is:
  *
  * - no owner;
  * - administrator only for a one time action to set parameters;
  * - no admin withdrawal;
  * - no rescue function;

 * - no emergency withdrawal;
 * - no configurable winner;
 * - no configurable payout destination;
 * - no configurable prize asset;
 *
 */
contract PrizeVault is ReentrancyGuard {
```

```
      using SafeERC20 for IERC20;


      // ========================================
      //              Custom Errors
      // ========================================


      error PrizeVault__ZeroAddress();
      error PrizeVault__ZeroAmount();
      error PrizeVault__InvalidProtocolContract();
      error PrizeVault__GameFinalized();
      error PrizeVault__GameNotFinalized();
      error PrizeVault__InvalidWinner();
      error PrizeVault__UnexpectedTransferAmount();


      // ========================================
      //               Immutables
      // ========================================


      // ERC20 WETH address
      IERC20 private immutable i_weth;


      // GameEngine adress
      address private immutable i_gameEngine;


      // ========================================
      //             Variables
      // ========================================


      bool public s_isFinalized; // Status to now if game has finalized
      uint256 private s_ethSnapshot; // Eth snapshot when game is finalized
      uint256 private s_wethSnapshot; // Weth when game is finalized
      uint256 private s_totalShares; // Total winning shares (e.g., N
survivors in Truce = N shares; 1 survivor = 1 share)
      uint256 private s_TotalClaimedShares; // To register the claimed
shares.


      mapping(address => uint256) private s_claimedShares; // Tracks how
many shares a winner address has already claimed

    // ========================================
    //                  Events
    // ========================================


    event Finalized(uint256 ethSnapshot, uint256 wethSnapshot, uint256
totalShares);
    event PrizeReleased(address indexed winner, uint256 ethAmount, uint256
wethAmount, uint256 sharesClaimed);


    // ========================================
    //                Constructor
    // ========================================


    constructor(address wethAddress_, address gameEngine_) {
          if (wethAddress_ == address(0) || gameEngine_ == address(0))
revert PrizeVault__ZeroAddress();
          i_weth = IERC20(wethAddress_);
          i_gameEngine = gameEngine_;
    }


    receive() external payable {
          if (s_isFinalized) revert PrizeVault__GameFinalized();
    }


    /**
    * @dev Function to create the snapshots of eth and weth for the
claimPrize
    * @param totalShares_ Total shares of the winners
    */
    function finalize(uint256 totalShares_) external {
          if (msg.sender != getGameEngineAddress()) revert
PrizeVault__InvalidProtocolContract();
          if (s_isFinalized) revert PrizeVault__GameFinalized();
          if (totalShares_ == 0) revert PrizeVault__ZeroAmount();


          s_isFinalized = true;
```

```
          s_totalShares = totalShares_;
          s_ethSnapshot = address(this).balance;
          s_wethSnapshot = i_weth.balanceOf(address(this));


          emit Finalized(s_ethSnapshot, s_wethSnapshot, totalShares_);
    }


    /**
    * @dev Function to claim prize if you are winner
    */

    function claimPrize() external nonReentrant {
          if (!s_isFinalized) revert PrizeVault__GameNotFinalized();


          // Query GameEngine to get total shares owned by msg.sender
          // (e.g. via an interface
IGameEngine(gameEngine).getWinningShares(msg.sender))
          uint256 userShares =
IGameEngine(getGameEngineAddress()).getWinningShares(msg.sender);
          if (userShares == 0) revert PrizeVault__InvalidWinner();


          uint256 unclaimedShares = userShares -
s_claimedShares[msg.sender];
          if (unclaimedShares == 0) revert PrizeVault__ZeroAmount();


          s_TotalClaimedShares += unclaimedShares;
          s_claimedShares[msg.sender] += unclaimedShares;


          // Calculate proportional in-kind payouts
          uint256 ethAmount = (s_ethSnapshot * unclaimedShares) /
s_totalShares;
          uint256 wethAmount = (s_wethSnapshot * unclaimedShares) /
s_totalShares;


          // Transfer ETH safely
          if (ethAmount > 0) {
              (bool success, ) = msg.sender.call{value: ethAmount}("");
              if (!success) revert PrizeVault__UnexpectedTransferAmount();
          }


          // Transfer WETH safely
          if(wethAmount > 0) {
              i_weth.safeTransfer(msg.sender, wethAmount);
          }


          emit PrizeReleased(msg.sender, ethAmount, wethAmount,
unclaimedShares);
    }


    // ========================================
    //                 View functions
    // ========================================


    /**
    * @dev Function to get WETH address
    */
    function getWethAddress() public view returns(address) {

           return address(i_weth);
     }


     /**
     * @dev Function to get the Game Engine address
     */
     function getGameEngineAddress() public view returns(address) {
           return i_gameEngine;
     }


     /**
     * @dev Function to get the Eth Snapshot
     */
     function getEthSnapshot() external view returns(uint256) {
           return s_ethSnapshot;
     }
```

```
    /**
    * @dev Function to get the Weth snapshot
    */
    function getWethSnapshot() external view returns(uint256) {
          return s_wethSnapshot;
    }


    /**
    * @dev Function to get Total Shares
    */
    function getTotalShares() external view returns(uint256) {
          return s_totalShares;
    }


    /**
    * @dev Function to get Total claimed shares
    */
    function getTotalClaimedShares() external view returns(uint256) {
          return s_TotalClaimedShares;
    }


}
```

PVGTreasury.sol

SOURCE REFERENCE FROM CARLOS'S ORIGINAL HANDOFF - canonical deltas above control if a deployment differs.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;


import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { SafeERC20 } from
"@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import { ReentrancyGuard } from
"@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import { Ownable } from "@openzeppelin/contracts/access/Ownable.sol";


/**
 * @title PVGTreasury
 * @author Carlos Gutiérrez
 * @notice Immutable secondary-market royalty splitter for
 * the Gluttons protocol.
 *
 * RESPONSIBILITIES
 * ----------------
 *
 * - Accept ETH: Needs a receive() external payable function so it can
catch incoming ETH from mints, gameplay actions,
 * and royalties.
 * - Access Control: Must have an owner or admin.
 * - Withdraw ETH: A restricted function withdrawETH(address to, uint256
amount) external onlyOwner that allows the team
 * to extract revenue
 * - Withdraw WETH: A restricted function withdrawWETH(address to, uint256
amount) external onlyOwner that uses SafeERC20
 * to transfer WETH out.
 *
 * RoyaltyTreasury does NOT:
 *
 * - Strict Security: Must not have any connection to PrizeVault or
GameEngine.
 *
 * ASSETS RECEIVED
 * ---------------
 *
 * The Treasury may receive:
 *
 * - native currency;
 * - ERC-20 WETH.
 *
 *
 * SECURITY MODEL
 * --------------

 *
 * There is:
 *
 * - owner for withdraw access;
 * - no withdrawal to arbitrary addresses;
 *
 */
contract PVGTreasury is ReentrancyGuard, Ownable {
      using SafeERC20 for IERC20;


      // ========================================
      //               Custom Errors
      // ========================================


      error PVGTreasury__ZeroAddress();
      error PVGTreasury__SameBeneficiary();
      error PVGTreasury__InvalidBeneficiary();
      error PVGTreasury__EmptyBalance();
      error PVGTreasury__NativeTransferFailed();
      error PVGTreasury__InvalidTransferAmount();
      error PVGTreasury__ZeroAmount();
      error PVGTreasury__PendingTransaction();
      error PVGTreasury__InvalidApprover();
      error PVGTreasury__NoPendingTransaction();
      error PVGTreasury__AlreadyApproved();
      error PVGTreasury__InvalidTransactionType();


      // ========================================
      //                 Structs
      // ========================================
```

```
      struct Transaction {
           address receiver;
           uint256 amount;
           uint8 currency; // 0 = eth, 1 = weth
           uint8 user1Approved;
           uint8 user2Approved;
      }


      // ========================================
      //               Immutables
      // ========================================


      // User 1 that can receive funds
      address private immutable i_user1;
      // user 2 that can rceive funds

    address private immutable i_user2;
    // IERC20 weth
    IERC20 private immutable i_weth;


    // ========================================
    //                Variables
    // ========================================


    // Current transaction id
    uint256 private s_id = 0;
    // Boolean to see if there is a pending transaction - just one
transaction can be set
    bool private s_pendingTransaction;
    // mapping of the transactions
    mapping(uint256 id => Transaction) private s_transaction;


    // ========================================
    //                  Events
    // ========================================


    // Event to register when eth has been withdrawn
    event EthWithdrawn(address indexed to, uint256 amount);
    // Event to register when weth has been withdrawn
    event WethWithdrawn(address indexed to, uint256 amount);
    // Event to show when a new transaction is requested
    event NewTransactionSet(address indexed receiver, uint256 amount);


    // ========================================
    //                Constructor
    // ========================================


    constructor(address owner_, address user1_, address user2_, address
weth_) Ownable(owner_) {
          if (owner_ == address(0) || user1_ == address(0) || user2_ ==
address(0)) revert PVGTreasury__ZeroAddress();
          if (user1_ == user2_) revert PVGTreasury__SameBeneficiary();
          i_user1 = user1_;
          i_user2 = user2_;
          i_weth = IERC20(weth_);
    }


    // ========================================
    //           Native Asset Reception
    // ========================================


    /**

    * @notice Accepts ordinary native-currency transfers.
    *
    * @dev
    * Some NFT marketplaces may settle royalties using
    * the chain's native currency.
    */
    receive() external payable {}


    /**
    * @notice Accepts native currency even when calldata
    * accompanies the payment.
    *
    * @dev
```

```
    * No calldata is interpreted or executed.
    */
    fallback() external payable {}


    function setTransactionApproval(address receiver_, uint256 amount_,
uint8 currency_) external onlyOwner {
          if (getPendingTransactionStatus()) revert
PVGTreasury__PendingTransaction();
          if (currency_ > 1) revert PVGTreasury__InvalidTransactionType();
          if (receiver_ != i_user1 && receiver_ != i_user2) revert
PVGTreasury__InvalidBeneficiary();
          if (amount_ == 0) revert PVGTreasury__ZeroAmount();


          Transaction storage transaction = s_transaction[getId()];
          if (transaction.receiver != address(0)) revert
PVGTreasury__PendingTransaction();
          transaction.amount = amount_;
          transaction.receiver = receiver_;
          transaction.currency = currency_;
          s_pendingTransaction = true;


          emit NewTransactionSet(receiver_, amount_);
    }


    function approveTransaction() external {
          if (msg.sender != i_user1 && msg.sender != i_user2) revert
PVGTreasury__InvalidApprover();
          if (!getPendingTransactionStatus()) revert
PVGTreasury__NoPendingTransaction();
          Transaction storage transaction = s_transaction[getId()];
          if (msg.sender == i_user1 && transaction.user1Approved == 1)
revert PVGTreasury__AlreadyApproved();

        if (msg.sender == i_user2 && transaction.user2Approved == 1)
revert PVGTreasury__AlreadyApproved();


        if (msg.sender == i_user1) {
            transaction.user1Approved = 1;
            if (transaction.user2Approved == 1) {
                   if (transaction.currency == 0) {
                       _withdrawETH(transaction.receiver,
transaction.amount);
                   } else {
                       _withdrawWeth(transaction.receiver,
transaction.amount);
                   }
            }
        }


        if (msg.sender == i_user2) {
            transaction.user2Approved = 1;
            if (transaction.user1Approved == 1) {
                   if (transaction.currency == 0) {
                       _withdrawETH(transaction.receiver,
transaction.amount);
                   } else {
                       _withdrawWeth(transaction.receiver,
transaction.amount);
                   }
            }
        }
    }


    function _withdrawETH(address to, uint256 amount) private nonReentrant
{
        if (to == address(0)) revert PVGTreasury__ZeroAddress();
        if (to != i_user1 && to != i_user2) revert
PVGTreasury__InvalidBeneficiary();
        if (amount == 0) revert PVGTreasury__ZeroAmount();


        uint256 contractBalance = getEthBalance();
        if (contractBalance == 0) revert PVGTreasury__EmptyBalance();
        if (amount > contractBalance) revert
PVGTreasury__InvalidTransferAmount();


        s_id ++;
        s_pendingTransaction = false;


        (bool success, ) = to.call{value: amount}("");
```

```
        if (!success) revert PVGTreasury__NativeTransferFailed();

        emit EthWithdrawn(to, amount);
    }

    function _withdrawWeth(address to, uint256 amount) private
nonReentrant {
        if (to == address(0)) revert PVGTreasury__ZeroAddress();
        if (to != i_user1 && to != i_user2) revert
PVGTreasury__InvalidBeneficiary();
        if (amount == 0) revert PVGTreasury__ZeroAmount();

        uint256 contractWethBalance = getWethBalance();
        if (contractWethBalance == 0) revert PVGTreasury__EmptyBalance();
        if (amount > contractWethBalance) revert
PVGTreasury__InvalidTransferAmount();

        s_id ++;
        s_pendingTransaction = false;

        i_weth.safeTransfer(to, amount);

        emit WethWithdrawn(to, amount);
    }

    // =======================================
    //              View functions
    // =======================================

    function getId() public view returns(uint256) {
        return s_id;
    }

    function getPendingTransactionStatus() public view returns(bool) {
        return s_pendingTransaction;
    }

    function getTransaction() public view returns(Transaction memory) {
        return s_transaction[s_id];
    }

    function getEthBalance() public view returns(uint256) {
        return address(this).balance;
    }

    function getWethBalance() public view returns(uint256) {

            return i_weth.balanceOf(address(this));
        }

       function getImmutables() external view returns(address, address,
 address) {
            return (i_user1, i_user2, address(i_weth));
        }

 }
```

RoyaltyTreasury.sol

SOURCE REFERENCE FROM CARLOS'S ORIGINAL HANDOFF - canonical deltas above control if a deployment differs.

```
// SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
 import {SafeERC20} from
 "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
 import {ReentrancyGuard} from
 "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
 import { IGameEngine } from "./interfaces/
```

IGameEngine.sol

SOURCE REFERENCE FROM CARLOS'S ORIGINAL HANDOFF - canonical deltas above control if a deployment differs.

```
// SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
 import {ReentrancyGuard} from
 "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
 import {IGluttonNFT} from "./interfaces/
```

IGluttonNFT.sol

SOURCE REFERENCE FROM CARLOS'S ORIGINAL HANDOFF - canonical deltas above control if a deployment differs.

```
";
 import {IPrizeVault} from "./interfaces/
```

IPrizeVault.sol

SOURCE REFERENCE FROM CARLOS'S ORIGINAL HANDOFF - canonical deltas above control if a deployment differs.

```
// SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 import { IGluttonNFT } from "./interfaces/IGluttonNFT.sol";
 import { IGameEngine } from "./interfaces/IGameEngine.sol";


 /**

 * @title Inspector
 * @author Carlos Gutiérrez
 * @notice Read-only aggregate for frontend display.
 *
 * Must do the following:
 * - Role: Exposes the canonical state before a user makes a purchase.
 * - Reads Only: It must have absolutely no state-changing functions (no
writes).
 * - No Authority: The frontend must never treat the Inspector as a source
of truth for executing transactions; it is purely for display and
aggregation.
 *
 */
contract Inspector {


      IGameEngine private immutable i_gameEngine;
      IGluttonNFT private immutable i_gluttonNFT;


      struct TokenStateView {
          address owner;
          uint8 visualState; // 0=UNREVEALED, 1=ALIVE, 2=FRESH, 3=ROTTEN
          uint64 expiry;
          bool isHungry;
      }


      struct GlobalState {
          uint256 aliveCount; // s_aliveCount()
          uint256 currentMealSeconds; // s_currentMealSeconds()
          bool isSettled; // isSettled()
          string currentPhase; // e.g., "FEAST", "PLAGUE", "LAST_SUPPER"
      }


      constructor(address gameEngine_, address gluttonNFT_) {
          i_gameEngine = IGameEngine(gameEngine_);
          i_gluttonNFT = IGluttonNFT(gluttonNFT_);
      }


      function getTokenView(uint256 tokenId) external view returns
(TokenStateView memory) {
          uint64 exp = i_gameEngine.effectiveExpiry(tokenId);
          uint8 vState = i_gameEngine.getVisualState(tokenId);


          bool hungry = false;


          // Only an ALIVE glutton can be hungry
          if (vState == 1) {

               if (exp <= block.timestamp) {
                   hungry = true; // It's a Faster at 0H or in Final Bite
               } else if (exp - block.timestamp <= 12 minutes) {
                   hungry = true; // Normal hunger threshold
               }
         }


         return TokenStateView({
               owner: i_gluttonNFT.ownerOf(tokenId),
               visualState: vState,
               expiry: exp,
               isHungry: hungry
         });
    }


    function getGlobalView() external view returns(GlobalState memory) {
```

```
        uint256 alive = i_gameEngine.s_aliveCount();
        uint256 meal = i_gameEngine.s_currentMealSeconds();
        bool settled = i_gameEngine.isSettled();
        uint256 S = i_gameEngine.S(); // Assuming GameEngine has an S()
getter


        string memory phase = "FEAST";


        if (settled) {
              phase = "SETTLED";
        } else if (alive > 0 && S > 0) {
              // Plague triggers at <= 50% population OR 2H meal floor
              if (alive <= (S * 50) / 100 || meal <= 120) {
                  phase = "PLAGUE";
              }
              // Last Supper triggers at <= 2.5% population (simplified for
view)
              if (alive <= (S * 25) / 1000) {
                  phase = "LAST_SUPPER";
              }
        } else if (i_gameEngine.s_gameStart() == 0) {
              phase = "PRE_GAME";
        }


        return GlobalState({
              aliveCount: alive,
              currentMealSeconds: meal,
              isSettled: settled,
              currentPhase: phase
        });

       }


 }



PrizeVault.sol

 // SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
 import {SafeERC20} from
 "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
 import {ReentrancyGuard} from
 "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
 import { IGameEngine } from "./interfaces/IGameEngine.sol";


 /**
  * @title PrizeVault
  * @author Carlos Gutiérrez
  * @notice Holds the active-game ETH and WETH prize pool.
  *
  * RESPONSIBILITIES
  * ----------------
  *
  * - Must accept incoming ETH.
  * - Must implement a pull-based claim system for winners to withdraw
 their share after the game is settled.
  * - Must distribute both ETH and WETH balances in-kind to the winners.
  * - Strict Security: The contract owner/admin must never be able to
 withdraw from this vault during the active game.
  *
  *
  * SECURITY MODEL
  * --------------
  *
  * There is:
  *
  * - no owner;
  * - administrator only for a one time action to set parameters;
  * - no admin withdrawal;
  * - no rescue function;

 * - no emergency withdrawal;
 * - no configurable winner;
 * - no configurable payout destination;
 * - no configurable prize asset;
 *
 */
contract PrizeVault is ReentrancyGuard {
```

```
      using SafeERC20 for IERC20;


      // ========================================
      //              Custom Errors
      // ========================================


      error PrizeVault__ZeroAddress();
      error PrizeVault__ZeroAmount();
      error PrizeVault__InvalidProtocolContract();
      error PrizeVault__GameFinalized();
      error PrizeVault__GameNotFinalized();
      error PrizeVault__InvalidWinner();
      error PrizeVault__UnexpectedTransferAmount();


      // ========================================
      //               Immutables
      // ========================================


      // ERC20 WETH address
      IERC20 private immutable i_weth;


      // GameEngine adress
      address private immutable i_gameEngine;


      // ========================================
      //             Variables
      // ========================================


      bool public s_isFinalized; // Status to now if game has finalized
      uint256 private s_ethSnapshot; // Eth snapshot when game is finalized
      uint256 private s_wethSnapshot; // Weth when game is finalized
      uint256 private s_totalShares; // Total winning shares (e.g., N
survivors in Truce = N shares; 1 survivor = 1 share)
      uint256 private s_TotalClaimedShares; // To register the claimed
shares.


      mapping(address => uint256) private s_claimedShares; // Tracks how
many shares a winner address has already claimed

    // ========================================
    //                  Events
    // ========================================


    event Finalized(uint256 ethSnapshot, uint256 wethSnapshot, uint256
totalShares);
    event PrizeReleased(address indexed winner, uint256 ethAmount, uint256
wethAmount, uint256 sharesClaimed);


    // ========================================
    //                Constructor
    // ========================================


    constructor(address wethAddress_, address gameEngine_) {
          if (wethAddress_ == address(0) || gameEngine_ == address(0))
revert PrizeVault__ZeroAddress();
          i_weth = IERC20(wethAddress_);
          i_gameEngine = gameEngine_;
    }


    receive() external payable {
          if (s_isFinalized) revert PrizeVault__GameFinalized();
    }


    /**
    * @dev Function to create the snapshots of eth and weth for the
claimPrize
    * @param totalShares_ Total shares of the winners
    */
    function finalize(uint256 totalShares_) external {
          if (msg.sender != getGameEngineAddress()) revert
PrizeVault__InvalidProtocolContract();
          if (s_isFinalized) revert PrizeVault__GameFinalized();
          if (totalShares_ == 0) revert PrizeVault__ZeroAmount();


          s_isFinalized = true;
```

```
          s_totalShares = totalShares_;
          s_ethSnapshot = address(this).balance;
          s_wethSnapshot = i_weth.balanceOf(address(this));


          emit Finalized(s_ethSnapshot, s_wethSnapshot, totalShares_);
    }


    /**
    * @dev Function to claim prize if you are winner
    */

    function claimPrize() external nonReentrant {
          if (!s_isFinalized) revert PrizeVault__GameNotFinalized();


          // Query GameEngine to get total shares owned by msg.sender
          // (e.g. via an interface
IGameEngine(gameEngine).getWinningShares(msg.sender))
          uint256 userShares =
IGameEngine(getGameEngineAddress()).getWinningShares(msg.sender);
          if (userShares == 0) revert PrizeVault__InvalidWinner();


          uint256 unclaimedShares = userShares -
s_claimedShares[msg.sender];
          if (unclaimedShares == 0) revert PrizeVault__ZeroAmount();


          s_TotalClaimedShares += unclaimedShares;
          s_claimedShares[msg.sender] += unclaimedShares;


          // Calculate proportional in-kind payouts
          uint256 ethAmount = (s_ethSnapshot * unclaimedShares) /
s_totalShares;
          uint256 wethAmount = (s_wethSnapshot * unclaimedShares) /
s_totalShares;


          // Transfer ETH safely
          if (ethAmount > 0) {
              (bool success, ) = msg.sender.call{value: ethAmount}("");
              if (!success) revert PrizeVault__UnexpectedTransferAmount();
          }


          // Transfer WETH safely
          if(wethAmount > 0) {
              i_weth.safeTransfer(msg.sender, wethAmount);
          }


          emit PrizeReleased(msg.sender, ethAmount, wethAmount,
unclaimedShares);
    }


    // ========================================
    //                 View functions
    // ========================================


    /**
    * @dev Function to get WETH address
    */
    function getWethAddress() public view returns(address) {

           return address(i_weth);
     }


     /**
     * @dev Function to get the Game Engine address
     */
     function getGameEngineAddress() public view returns(address) {
           return i_gameEngine;
     }


     /**
     * @dev Function to get the Eth Snapshot
     */
     function getEthSnapshot() external view returns(uint256) {
           return s_ethSnapshot;
     }
```

```
      /**
      * @dev Function to get the Weth snapshot
      */
      function getWethSnapshot() external view returns(uint256) {
            return s_wethSnapshot;
      }


      /**
      * @dev Function to get Total Shares
      */
      function getTotalShares() external view returns(uint256) {
            return s_totalShares;
      }


      /**
      * @dev Function to get Total claimed shares
      */
      function getTotalClaimedShares() external view returns(uint256) {
            return s_TotalClaimedShares;
      }


 }



PVGTreasury.sol // works for PVGTreasury and FutureRewardsVault

// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;


import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { SafeERC20 } from
"@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import { ReentrancyGuard } from
"@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import { Ownable } from "@openzeppelin/contracts/access/Ownable.sol";


/**
 * @title PVGTreasury
 * @author Carlos Gutiérrez
 * @notice Immutable secondary-market royalty splitter for
 * the Gluttons protocol.
 *
 * RESPONSIBILITIES
 * ----------------
 *
 * - Accept ETH: Needs a receive() external payable function so it can
catch incoming ETH from mints, gameplay actions,
 * and royalties.
 * - Access Control: Must have an owner or admin.
 * - Withdraw ETH: A restricted function withdrawETH(address to, uint256
amount) external onlyOwner that allows the team
 * to extract revenue
 * - Withdraw WETH: A restricted function withdrawWETH(address to, uint256
amount) external onlyOwner that uses SafeERC20
 * to transfer WETH out.
 *
 * RoyaltyTreasury does NOT:
 *
 * - Strict Security: Must not have any connection to PrizeVault or
GameEngine.
 *
 * ASSETS RECEIVED
 * ---------------
 *
 * The Treasury may receive:
 *
 * - native currency;
 * - ERC-20 WETH.
 *
 *
 * SECURITY MODEL
 * --------------

 *
 * There is:
 *
 * - owner for withdraw access;
 * - no withdrawal to arbitrary addresses;
 *
 */
contract PVGTreasury is ReentrancyGuard, Ownable {
      using SafeERC20 for IERC20;
```

```
      // ========================================
      //              Custom Errors
      // ========================================


      error PVGTreasury__ZeroAddress();
      error PVGTreasury__SameBeneficiary();
      error PVGTreasury__InvalidBeneficiary();
      error PVGTreasury__EmptyBalance();
      error PVGTreasury__NativeTransferFailed();
      error PVGTreasury__InvalidTransferAmount();
      error PVGTreasury__ZeroAmount();
      error PVGTreasury__PendingTransaction();
      error PVGTreasury__InvalidApprover();
      error PVGTreasury__NoPendingTransaction();
      error PVGTreasury__AlreadyApproved();
      error PVGTreasury__InvalidTransactionType();


      // ========================================
      //                 Structs
      // ========================================


      struct Transaction {
           address receiver;
           uint256 amount;
           uint8 currency; // 0 = eth, 1 = weth
           uint8 user1Approved;
           uint8 user2Approved;
      }


      // ========================================
      //              Immutables
      // ========================================


      // User 1 that can receive funds
      address private immutable i_user1;
      // user 2 that can rceive funds

    address private immutable i_user2;
    // IERC20 weth
    IERC20 private immutable i_weth;


    // ========================================
    //               Variables
    // ========================================


    // Current transaction id
    uint256 private s_id = 0;
    // Boolean to see if there is a pending transaction - just one
transaction can be set
    bool private s_pendingTransaction;
    // mapping of the transactions
    mapping(uint256 id => Transaction) private s_transaction;


    // ========================================
    //                 Events
    // ========================================


    // Event to register when eth has been withdrawn
    event EthWithdrawn(address indexed to, uint256 amount);
    // Event to register when weth has been withdrawn
    event WethWithdrawn(address indexed to, uint256 amount);
    // Event to show when a new transaction is requested
    event NewTransactionSet(address indexed receiver, uint256 amount);


    // ========================================
    //               Constructor
    // ========================================


    constructor(address owner_, address user1_, address user2_, address
weth_) Ownable(owner_) {
          if (owner_ == address(0) || user1_ == address(0) || user2_ ==
address(0)) revert PVGTreasury__ZeroAddress();
          if (user1_ == user2_) revert PVGTreasury__SameBeneficiary();
          i_user1 = user1_;
          i_user2 = user2_;
          i_weth = IERC20(weth_);
```

```
    }


    // ======================================
    //           Native Asset Reception
    // ======================================


    /**

    * @notice Accepts ordinary native-currency transfers.
    *
    * @dev
    * Some NFT marketplaces may settle royalties using
    * the chain's native currency.
    */
    receive() external payable {}


    /**
    * @notice Accepts native currency even when calldata
    * accompanies the payment.
    *
    * @dev
    * No calldata is interpreted or executed.
    */
    fallback() external payable {}


    function setTransactionApproval(address receiver_, uint256 amount_,
uint8 currency_) external onlyOwner {
          if (getPendingTransactionStatus()) revert
PVGTreasury__PendingTransaction();
          if (currency_ > 1) revert PVGTreasury__InvalidTransactionType();
          if (receiver_ != i_user1 && receiver_ != i_user2) revert
PVGTreasury__InvalidBeneficiary();
          if (amount_ == 0) revert PVGTreasury__ZeroAmount();


          Transaction storage transaction = s_transaction[getId()];
          if (transaction.receiver != address(0)) revert
PVGTreasury__PendingTransaction();
          transaction.amount = amount_;
          transaction.receiver = receiver_;
          transaction.currency = currency_;
          s_pendingTransaction = true;


          emit NewTransactionSet(receiver_, amount_);
    }


    function approveTransaction() external {
          if (msg.sender != i_user1 && msg.sender != i_user2) revert
PVGTreasury__InvalidApprover();
          if (!getPendingTransactionStatus()) revert
PVGTreasury__NoPendingTransaction();
          Transaction storage transaction = s_transaction[getId()];
          if (msg.sender == i_user1 && transaction.user1Approved == 1)
revert PVGTreasury__AlreadyApproved();

        if (msg.sender == i_user2 && transaction.user2Approved == 1)
revert PVGTreasury__AlreadyApproved();


        if (msg.sender == i_user1) {
            transaction.user1Approved = 1;
            if (transaction.user2Approved == 1) {
                  if (transaction.currency == 0) {
                      _withdrawETH(transaction.receiver,
transaction.amount);
                  } else {
                      _withdrawWeth(transaction.receiver,
transaction.amount);
                  }
            }
        }


        if (msg.sender == i_user2) {
            transaction.user2Approved = 1;
            if (transaction.user1Approved == 1) {
                  if (transaction.currency == 0) {
                      _withdrawETH(transaction.receiver,
transaction.amount);
                  } else {
                      _withdrawWeth(transaction.receiver,
transaction.amount);
                  }
```

```
            }
        }
    }


    function _withdrawETH(address to, uint256 amount) private nonReentrant
{
        if (to == address(0)) revert PVGTreasury__ZeroAddress();
        if (to != i_user1 && to != i_user2) revert
PVGTreasury__InvalidBeneficiary();
        if (amount == 0) revert PVGTreasury__ZeroAmount();


        uint256 contractBalance = getEthBalance();
        if (contractBalance == 0) revert PVGTreasury__EmptyBalance();
        if (amount > contractBalance) revert
PVGTreasury__InvalidTransferAmount();


        s_id ++;
        s_pendingTransaction = false;


        (bool success, ) = to.call{value: amount}("");

         if (!success) revert PVGTreasury__NativeTransferFailed();


         emit EthWithdrawn(to, amount);
    }


    function _withdrawWeth(address to, uint256 amount) private
nonReentrant {
         if (to == address(0)) revert PVGTreasury__ZeroAddress();
         if (to != i_user1 && to != i_user2) revert
PVGTreasury__InvalidBeneficiary();
         if (amount == 0) revert PVGTreasury__ZeroAmount();


         uint256 contractWethBalance = getWethBalance();
         if (contractWethBalance == 0) revert PVGTreasury__EmptyBalance();
         if (amount > contractWethBalance) revert
PVGTreasury__InvalidTransferAmount();


         s_id ++;
         s_pendingTransaction = false;


         i_weth.safeTransfer(to, amount);


         emit WethWithdrawn(to, amount);
    }


    // ========================================
    //              View functions
    // ========================================


    function getId() public view returns(uint256) {
         return s_id;
    }


    function getPendingTransactionStatus() public view returns(bool) {
         return s_pendingTransaction;
    }


    function getTransaction() public view returns(Transaction memory) {
         return s_transaction[s_id];
    }


    function getEthBalance() public view returns(uint256) {
         return address(this).balance;
    }


    function getWethBalance() public view returns(uint256) {

             return i_weth.balanceOf(address(this));
        }


        function getImmutables() external view returns(address, address,
```

```
 address) {
            return (i_user1, i_user2, address(i_weth));
        }


 }
```

RoyaltyTreasury.sol

```
 // SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
 import {SafeERC20} from
 "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
 import {ReentrancyGuard} from
 "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
 import { IGameEngine } from "./interfaces/IGameEngine.sol";


 /**
  * @title RoyaltyTreasury
  * @notice Routing engine for OpenSea's 4% creator earnings
  *
  */
 contract RoyaltyTreasury is ReentrancyGuard {
        using SafeERC20 for IERC20;


        // ========================================
        //               Custom Errors
        // ========================================


        error RoyaltyTreasury__ZeroAddress();
        error RoyaltyTreasury__TransferFailed();
        error RoyaltyTreasury__ZeroAmount();


        // ========================================
        //                Constants
        // ========================================

    // Parts to split share
    uint8 private constant PARTS = 8;


    // ========================================
    //                   Immutables
    // ========================================


    IERC20 private immutable i_weth;
    address private immutable i_pvgTreasury;
    address private immutable i_prizeVault;
    address private immutable i_futureRewardsVault;
    address private immutable i_gameEngine;


    // ========================================
    //                  Constructor
    // ========================================


    /**
    * @dev Constructor params
    * @param weth_ WETH address
    * @param pvgTreasury_ PVGTreasury address
    * @param prizeVault_ PrizeVault address
    * @param futureRewardsVault_ FutureRewardsVault address
    * @param gameEngine_ Game Engine address
    */
    constructor(address weth_, address pvgTreasury_, address prizeVault_,
address futureRewardsVault_, address gameEngine_) {
          if (
                weth_ == address(0) || pvgTreasury_ == address(0) ||
prizeVault_ == address(0) || futureRewardsVault_ == address(0)
                || gameEngine_ == address(0)
          ) revert RoyaltyTreasury__ZeroAddress();


          i_weth = IERC20(weth_);
          i_pvgTreasury = pvgTreasury_;
          i_prizeVault = prizeVault_;
          i_futureRewardsVault = futureRewardsVault_;
```

```
        i_gameEngine = gameEngine_;
    }


    receive() external payable {
        transferEth();
    }


    fallback() external payable {

        transferEth();
    }


    /**
    * @dev Function to transfer Ether received
    */
    function transferEth() public payable {
        if (msg.value <= 0) revert RoyaltyTreasury__ZeroAmount();


        uint256 pvgShare = msg.value / PARTS;
        uint256 playerShares = msg.value - pvgShare;


        bool gameSettled = IGameEngine(i_gameEngine).isSettled();


        if (!gameSettled) {
            (bool prizeVaultSuccess, ) = i_prizeVault.call{value:
playerShares}("");
            if (!prizeVaultSuccess) revert
RoyaltyTreasury__TransferFailed();
        } else {
            (bool futureRewardsSuccess, ) =
i_futureRewardsVault.call{value: playerShares}("");
            if (!futureRewardsSuccess) revert
RoyaltyTreasury__TransferFailed();
        }


        (bool success, ) = i_pvgTreasury.call{value: pvgShare}("");
        if (!success) revert RoyaltyTreasury__TransferFailed();
    }


    /**
    * @dev Function to send WETH to the respective smart contracts,
anyone can call the function
    */
    function flushWETH() external {
        uint256 wethContractAmount = i_weth.balanceOf(address(this));


        if (wethContractAmount == 0) revert RoyaltyTreasury__ZeroAmount();


        uint256 pvgShare = wethContractAmount / PARTS;
        uint256 playerShares = wethContractAmount - pvgShare;


        bool gameSettled = IGameEngine(i_gameEngine).isSettled();


        if (!gameSettled) {
            i_weth.safeTransfer(i_prizeVault, playerShares);

          } else {
              i_weth.safeTransfer(i_futureRewardsVault, playerShares);
          }


          i_weth.safeTransfer(i_pvgTreasury, pvgShare);
      }


}
```

IGameEngine.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;


interface IGameEngine {
```

```
        // 0 = UNREVEALED, 1 = ALIVE/FASTING/FINAL_BITE/SETTLED,
 2 = FRESH CORPSE, 3 = ROTTEN
        function getVisualState(uint256 tokenId) external view
 returns (uint8);


        function isSettled() external view returns (bool);


        function getWinningShares(address account) external view
 returns (uint256);


        function s_aliveCount() external view returns (uint256);


        function effectiveExpiry(uint256 tokenId) external view
 returns (uint64);


        function s_currentMealSeconds() external view
 returns(uint256);


        function S() external view returns(uint256);

        function s_gameStart() external view returns(uint64);


 }
```

IGluttonNFT.sol

```
 // SPDX-License-Identifier: MIT
 pragma solidity 0.8.30;


 interface IGluttonNFT {


        function gameMint(address to, uint256 amount) external;


        function gameBurn(uint256 tokenId) external;


        function ownerOf(uint256 tokenId) external view returns
 (address owner);


        function balanceOf(address owner) external view returns
 (uint256);


 }
```

IPrizeVault.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;


interface IPrizeVault {


       function finalize(uint256 totalShares_) external;


}
```

23. FINAL HANDOFF RULE

Carlos should build the frontend against the actual deployed ABI/address set, then use this manual to determine what to render, when to render it, and how to recover from RPC/state-sync failures.

Do not copy a stale constant from a PDF into React. Read deployment state wherever the contract exposes it. In particular, do not hardcode 100 or 2,000 for inventory discovery after Game Start; use Starting Population S / deployed supply.

If a deployed contract disagrees with the canonical gameplay documents, stop and reconcile the contract before hiding the mismatch in frontend code.